

7 web development

Putting it all together



Sooner or later, you'll want to share your app with lots of people.

You have many options for doing this. Pop your code on PyPI, send out lots of emails, put your code on a CD or USB, or simply install your app manually on the computers of those people who need it. Sounds like a lot of work...not to mention boring. Also, what happens when you produce the next best version of your code? What happens then? How do you manage the update? Let's face it: it's such a pain that you'll think up really creative excuses not to. Luckily, you don't have to do any of this: just create a webapp instead. And, as this chapter demonstrates, using Python for web development is a breeze.

It's good to share



You're a victim of your own success.

The new requests come flooding in right after Coach Kelly starts showing off your latest program. It appears that everyone wants access to the coach's data!

The thing is: what's the "best way" to do this?

You can put your program on the Web



A “webapp” is what you want.

If you develop your program as a *Web-based application* (or *webapp*, for short), your program is:

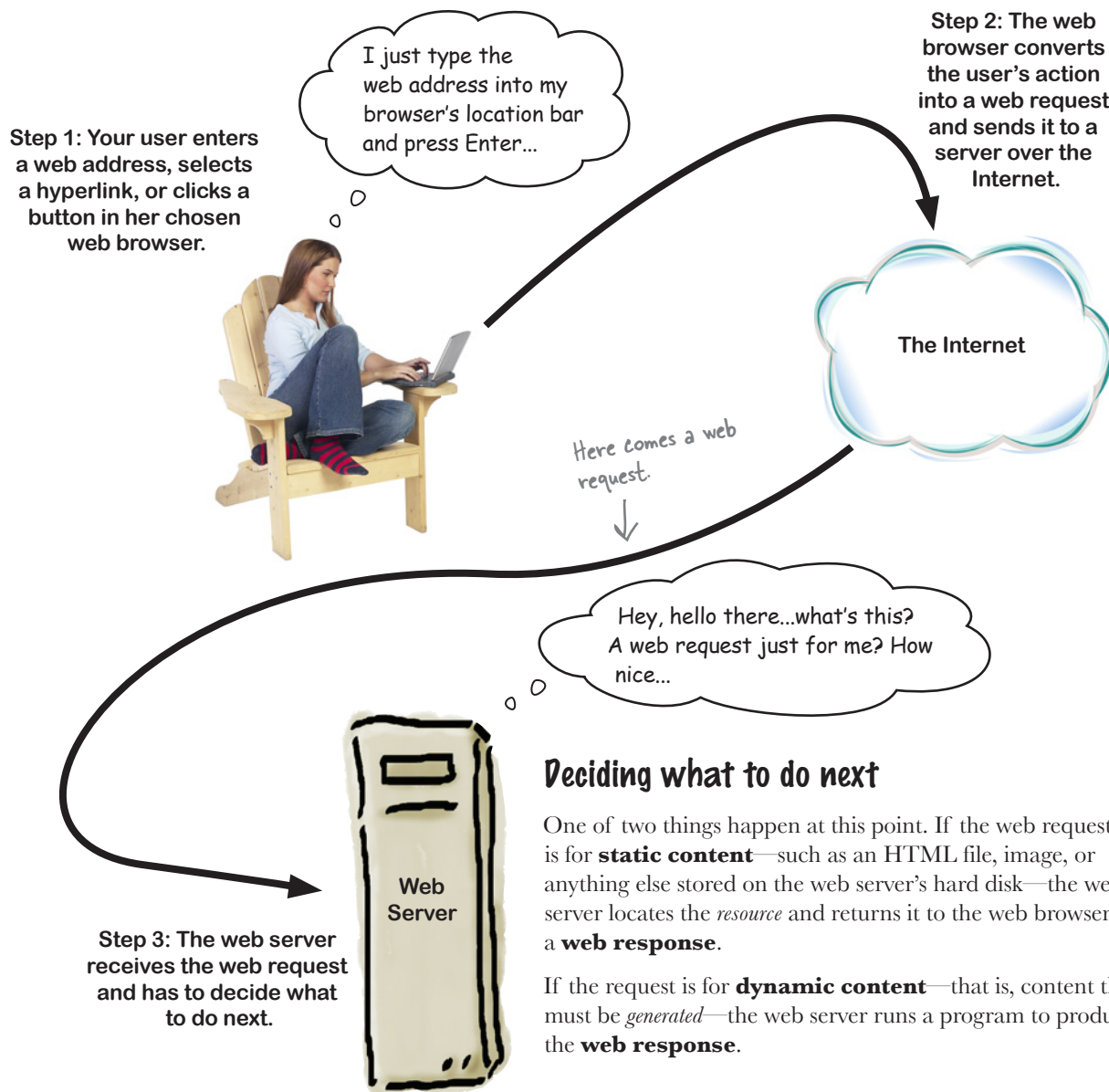
- Available to everyone who can get to your website
- In one place on your web server
- Easy to upate as new functionality is needed

But...how do webapps actually work?



Webapps Up Close

No matter what you do on the Web, it's all about *requests* and *responses*. A **web request** is sent from a web browser to a web server as the result of some user interaction. On the web server, a **web response** (or *reply*) is formulated and *sent back* to the web browser. The entire process can be summarized in five steps.



Step 4: The web server processes the web request, creating a web response, which is sent back over the Internet to the waiting web browser.



Here you go...a web response generated just for you. Enjoy!

Here comes a web response.

The Internet

The (potentially) many substeps of step 4

In practice, step 4 can involve multiple *substeps*, depending on what the web server has to do to produce the response. Obviously, if all the server has to do is locate static content and sent it back to the browser, the substeps aren't too taxing, because it's all just file I/O.

However, when dynamic content must be *generated*, the substeps involve the web server *locating* the program to execute, *executing* the located program, and then *capturing* the output from the program as the web response...which is then sent back to the waiting web browser.

This dynamic content generation process has been standardized since the early days of the Web and is known as the **Common Gateway Interface (CGI)**. Programs that conform to the standard are often referred to as *CGI scripts*.

That's exactly what I need. Thanks!

Step 5: The web browser receives the web response and displays it on your user's screen.



What does your webapp need to do?

Let's take a moment to consider what you want your webapp to look like and how it should behave on your user's web browser. You can then use this information to help you specify what your webapp needs to do.



Sharpen your pencil



There's nothing like grabbing your pencil and a few blank paper napkins to quickly sketch a simple web design. You probably need three web pages: a "welcome" page, a "select an athlete" page, and a "display times" page. Go ahead and draw out a rough design on the napkins on this page, and don't forget to draw any linkages between the pages (where it makes sense).



Sharpen your pencil Solution

The home page displays a friendly graphic and a link to start the web app.



Welcome to Coach Kelly's Website
For now, all that you'll find here is my athlete's timing data.
See you on the track!

There's nothing like grabbing your pencil and a few blank paper napkins to quickly sketch a simple web design. You probably need three web pages: a "welcome" page, a "select an athlete" page, and a "display times" page. You were to draw out a rough design on the napkins. You were to draw any linkages between the pages (where it made sense).

Click on the home page's link to go to a page that displays a list of all the coach's athletes. Click on an athlete's radio button and then the "Select" button to see the data.

Select an athlete from this list to work with:

- ☒ Sarah
- ☐ James
- ☐ Julie
- ☐ Mikey

Select

Timing data for Sarah:

2.18
2.21
2.22

The third web page displays the selected athlete's data and provides links back to the other two pages.

Home

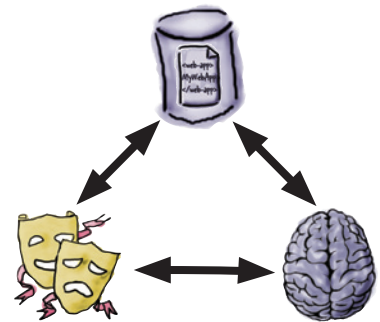
Select another athlete.

Design your webapp with MVC

Now that you have an idea of the pages your webapp needs to provide, your next question should be: *what's the best way to build this thing?*

Ask 10 web developers that question and you'll get 10 different answers; the answer often depends on whom you ask.

Despite this, the general consensus is that great webapps conform to the *Model-View-Controller* pattern, which helps you segment your webapp's code into easily manageable functional chunks (or *components*):



The Model

The code to store (and sometimes process) your webapp's data



The View

The code to format and display your webapp's user interface(s)



The Controller

The code to glue your webapp together and provide its business logic

By following the MVC pattern, you build your webapp in such a way as to enable your webapp to grow as new requirements dictate. You also open up the possibility of splitting the workload among a number of people, one for each component.

Let's build each of the MVC components for your webapp.

Model your data

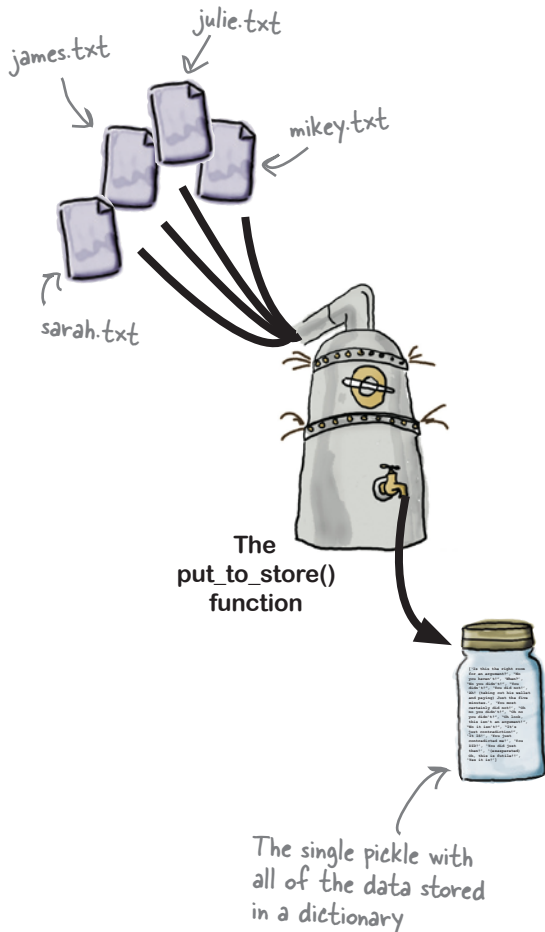
Your web server needs to store a single copy of your data, which in this case is Coach Kelly's timing values (which start out in his text files).

When your webapp starts, the data in the text files needs to be converted to `AthleteList` object instances, stored within a dictionary (indexed by athlete name), and then saved as a pickle file. Let's put this functionality in a new function called `put_to_store()`.

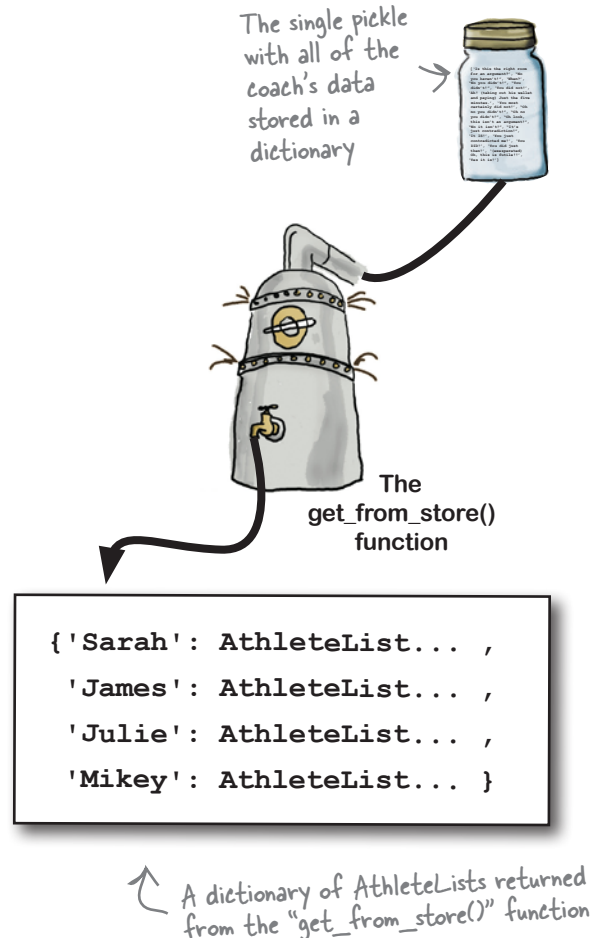
While your webapp runs, the data in the pickle needs to be available to your webapp as a dictionary. Let's put this functionality in another new function called `get_from_store()`.



When your webapp starts:



While your webapp runs:





Exercise

Here is the outline for a new module called `athletemodel.py`, which provides the functionality described on the previous page. Some of the code is already provided for you. Your job is to provide the rest of the code to the `put_to_store()` and `get_from_store()` functions. Don't forget to protect any file I/O calls.

```
import pickle
from athletemodel import AthleteList
```

```
def get_coach_data(filename):
    # Not shown here as it has not changed since the last chapter.
```

```
def put_to_store(files_list):
    all_athletes = {}
```

This function is called with a list of filenames as its sole argument.

You need code in here to populate the dictionary with the data from the files.

And don't forget to save the dictionary to a pickle (and check for file I/O errors).

```
    return(all_athletes)
```

```
def get_from_store():
    all_athletes = {}
```

Get the dictionary from the file, so that it can be returned to the caller.

Both functions need to return a dictionary of AthleteLists.

```
    return(all_athletes)
```



Exercise Solution

Here is the outline for a new module called `athletemodel.py`, which provides the functionality described on the previous page. Some of the code is already provided for you. Your job was to provide the rest of the code to the `put_to_store()` and `get_from_store()` functions. You were not to forget to protect any file I/O calls.

```
import pickle
from athletemodel import AthleteList
```

```
def get_coach_data(filename):
    # Not shown here as it has not changed since the last chapter.
```

```
def put_to_store(files_list):
```

```
    all_athletes = {}
```

```
    for each_file in files_list:
```

```
        ath = get_coach_data(each_file)
```

```
        all_athletes[ath.name] = ath
```

```
    try:
```

```
        with open('athletes.pickle', 'wb') as athf:
```

```
            pickle.dump(all_athletes, athf)
```

```
    except IOError as ioerr:
```

```
        print('File error (put_and_store): ' + str(ioerr))
```

```
    return(all_athletes)
```

```
def get_from_store():
```

```
    all_athletes = {}
```

```
    try:
```

```
        with open('athletes.pickle', 'rb') as athf:
```

```
            all_athletes = pickle.load(athf)
```

```
    except IOError as ioerr:
```

```
        print('File error (get_from_store): ' + str(ioerr))
```

```
    return(all_athletes)
```

Take each file, turn it into an `AthleteList` object instance, and add the athlete's data to the dictionary.

Each athlete's name is used as the "key" in the dictionary. The "value" is the `AthleteList` object instance.

Save the entire dictionary of `AthleteList`s to a pickle.

And don't forget a try/except to protect your file I/O code.

Simply read the entire pickle into the dictionary. What could be easier?

Again...don't forget your try/except.



An IDLE Session

Let's test your code to ensure that it is working to specification. Type your code into an IDLE edit window and save your code into a folder that also includes the coach's text files. Press F5 to import your code to the IDLE shell, and then use the `dir()` command to confirm that the import has been successful:

```
>>> dir()

['AthleteList', '__builtins__', '__doc__', '__name__', '__package__', 'get_coach_data',
'get_from_store', 'pickle', 'put_to_store']
```

Create a list of files to work with, and then call the `put_to_store()` function to take the data in the list of files and turn them into a dictionary stored in a pickle:

```
>>> the_files = ['sarah.txt', 'james.txt', 'mikey.txt', 'julie.txt']
>>> data = put_to_store(the_files)
>>> data

{'James Lee': ['2-34', '3:21', '2.34', '2.45', '3.01', '2:01', '2:01', '3:10', '2-22', '2-01', '2.01', '2:16'], 'Sarah Sweeney': ['2:58', '2.58', '2:39', '2-25', '2-55', '2:54', '2.18', '2:55', '2:55', '2:22', '2-21', '2.22'], 'Julie Jones': ['2.59', '2.11', '2:11', '2:23', '3-10', '2-23', '3:10', '3.21', '3-21', '3.01', '3.02', '2:59'], 'Mikey McManus': ['2:22', '3.01', '3:01', '3.02', '3:02', '3.02', '3:22', '2.49', '2:38', '2:40', '2.22', '2-31']}
```

Here's all of the AthleteLists.

At this point, the `athletes.pickle` file should appear in the same folder as your code and text files. Recall that this file is a *binary* file, so trying to view it in IDLE or in your editor is not going to make much sense. To access the data, use the dictionary returned by the `put_to_store()` or `get_from_store()` functions.

Use the existing data in the `data` dictionary to display each athlete's name and date of birth:

```
>>> for each_athlete in data:
    print(data[each_athlete].name + ' ' + data[each_athlete].dob)
```

```
James Lee 2002-3-14
Sarah Sweeney 2002-6-17
Julie Jones 2002-8-17
Mikey McManus 2002-2-24
```

By accessing the "name" and "dob" attributes, you can get at the rest of the AthleteList data.

Use the `get_from_store()` function to load the pickled data into another dictionary, then confirm that the results are as expected by repeating the code to display each athlete's name and date of birth:

```
>>> data_copy = get_from_store()
>>> for each_athlete in data_copy:
    print(data_copy[each_athlete].name + ' ' + data_copy[each_athlete].dob)
```

```
James Lee 2002-3-14
Sarah Sweeney 2002-6-17
Julie Jones 2002-8-17
Mikey McManus 2002-2-24
```

The data in the returned dictionary is as expected, exactly the same as that produced by `put_to_store()`.

View your interface

With your model code written and working, it's time to look at your view code, which creates your webapp's user interface (UI).

On the Web, UIs are created with HTML, the Web's markup technology. If you are new to HTML, it is worth taking some time to become familiar with this *critical* web development technology. There's lots of material on the Web and more than a few good books out there.



[Note from Marketing:
This is the book that we
recommend for quickly
getting up to speed with
HTML...not that we're
biased or anything. 😊 J.]

Hey, we hear you are getting into
web development? We have a small
module that we put together that might help
you generate HTML. It's a little rough, but it
works. You're more than welcome to use it for
your projects, if you like.

(Most of) the Head First
Code Review Team



YATE: Yet Another Template Engine

Your friends over at the Head First Code Review Team heard you're planning to write some code to generate HTML for your webapp's UI. They've sent over some code that they swear will make your life easier. It's a small library of HTML-generating helper functions called `yate`. The code was produced quickly and was originally designed to be "throw away," so the team has provided it *as is*. It's somewhat raw, but it should be OK.



LONG Exercise

Let's get to know the `yate` code before proceeding with the rest of this chapter. For each chunk of code presented, provide a written description of what you think it does in the spaces provided:

Take a moment to look up the "Template" module in Python's documentation set.

Write your explanations in the spaces.

```
from string import Template
```

```
def start_response(resp="text/html"):
    return('Content-type: ' + resp + '\n\n')
```

One has already been done for you.

This function takes a single (optional) string as its argument and uses it to create a CGI "Content-type:" line, with "text/html" as the default.

```
def include_header(the_title):
    with open('templates/header.html') as headf:
        head_text = headf.read()
    header = Template(head_text)
    return(header.substitute(title=the_title))
```

```
def include_footer(the_links):  
    with open('templates/footer.html') as footf:  
        foot_text = footf.read()  
  
    link_string = ''  
  
    for key in the_links:  
        link_string += '<a href="' + the_links[key] + '"'> + key +  
                        '</a>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;'<br>  
    footer = Template(foot_text)  
    return(footer.substitute(links=link_string))
```

```

def start_form(the_url, form_type="POST"):
    return('<form action="' + the_url + '" method="' + form_type + '">')
    .....
    .....

def end_form(submit_msg="Submit"):
    return('<p></p><input type=submit value="' + submit_msg + '"></form>')
    .....
    .....

def radio_button(rb_name, rb_value):
    return('<input type="radio" name="' + rb_name +
           '" value="' + rb_value + '"> ' + rb_value + '<br />')
    .....
    .....

def u_list(items):
    u_string = '<ul>'
    for item in items:
        u_string += '<li>' + item + '</li>'
    u_string += '</ul>'
    return(u_string)
    .....
    .....
    .....

def header(header_text, header_level=2):
    return('<h' + str(header_level) + '>' + header_text +
           '</h' + str(header_level) + '>')
    .....
    .....
    .....

def para(para_text):
    return('<p>' + para_text + '</p>')
    .....
    .....

```

Let's get to know the `yate` code before proceeding with the rest of this chapter. For each chunk of code presented, you were to provide a written description of what you think it does:

Import the "Template" class from the standard library's "string" module. This allows for simple string-substitution templates.

```
def start_response(resp="text/html"):  
    return('Content-type: ' + resp + '\n\n')
```

This function takes a single (optional) string as its argument and uses it to create a CGI "Content-type:" line, with "text/html" as the default.

```
def include_header(the_title):
    with open('templates/header.html') as headf:
        head_text = headf.read()
        header = Template(head_text)
    return(header.substitute(title=the_title))
```

This function takes a single string as its argument and uses it as the title for the start of a HTML page. The page itself is stored within a separate file in "templates/header.html", and the title is substituted in as needed.

[illegible]

This looks a little weird, but it's an HTML hack for forcing spaces into a string.

Turn the dictionary of links into a string, which is then substituted into the template.

Similar to the "include_header" function, this one uses its single string as its argument to create the end of a HTML page. The page itself is stored within a separate file in "templates/footer.html", and the argument is used to dynamically create a set of HTML link tags. Based on how they are used, it looks like the argument needs to be a dictionary.


 This is typically either "POST" or "GET".

```
def start_form(the_url, form_type="POST"):
    return('<form action="' + the_url + '" method="' + form_type + '">')
```

.....

This function returns the HTML for the start of a form and lets the caller specify the URL to send the form's data to, as well as the method to use.

.....

```
def end_form(submit_msg="Submit"):
    return('<p></p><input type=submit value="' + submit_msg + '"></form>')
```

.....

This function returns the HTML markup, which terminates the form while allowing the caller to customize the text of the form's "submit" button.

.....

```
def radio_button(rb_name, rb_value):
    return('<input type="radio" name="' + rb_name +
           '" value="' + rb_value + '"> ' + rb_value + '<br />')
```

.....

Given a radio-button name and value, create a HTML radio button (which is typically included within a HTML form). Note: both arguments are required.

.....

```
def u_list(items):
    u_string = '<ul>'
    for item in items:
        u_string += '<li>' + item + '</li>'
    u_string += '</ul>'
    return(u_string)
```

A simple "for" loop does the trick.



.....

Given a list of items, this function turns the list into a HTML unnumbered list. A simple "for" loop does all the work, adding a LI to the UL element with each iteration.

.....

```
def header(header_text, header_level=2):
    return('<h' + str(header_level) + '>' + header_text +
           '</h' + str(header_level) + '>')
```

.....

Create and return a HTML header tag (H1, H2, H2, and so on) with level 2 as the default. The "header_text" argument is required.

.....

```
def para(para_text):
    return('<p>' + para_text + '</p>')
```

.....

Enclose a paragraph of text (a string) in HTML paragraph tags. Almost not worth the effort, is it?

.....

there are no Dumb Questions

Q: Where are the HTML templates used in the `include_header()` and `include_footer()` functions?

A: They are included with the `yate` module's download. Go ahead and grab them from the *Head First Python* support website, and put them into a folder of your choice.

Q: Why do I need `yate` at all? Why not include the HTML that I need right in the code and generate it with `print()` as needed?

A: You could, but it's not as flexible as the approach shown here. And (speaking from bitter experience) using a collection of `print()` statements to generate HTML works, but it turns your code into an *unholy mess*.

Q: And you did this because you are using MVC?

A: Partly, yes. The reason the MVC pattern is being followed is to ensure that the model code is separate from the view code, which are both separate from the controller code. No matter the size of the project, following MVC can make your life easier.

Q: But surely MVC is overkill for something this small?

A: We don't think so, because you can bet that your webapp will grow, and when you need to add more features, the MVC "separation of duties" really shines.



An IDLE Session

Let's get to know the `yate` module even more. With the code downloaded and tucked away in an easy-to-find folder, load the module into IDLE and press F5 to take it for a spin. Let's start by testing the `start_response()` function. The CGI standard states that every web response must start with a header line that indicates the *type* of the data included in the request, which `start_response()` lets you control:

```
>>> start_response()
'Content-type: text/html\n\n'
>>> start_response("text/plain")
'Content-type: text/plain\n\n'
>>> start_response("application/json")
'Content-type: application/json\n\n'
```

The default CGI response header, plus variations on a theme.

The `include_header()` function generates the start of a web page and let's you customize its title:

```
>>> include_header("Welcome to my home on the web!")
'<html>\n<head>\n<title>Welcome to my home on the web!</title>\n<link type="text/css"
rel="stylesheet" href="/coach.css" />\n</head>\n<body>\n<h1>Welcome to my home on the web!</
h1>\n'
```

This all looks a little bit messy, but don't worry; it's meant to be processed by your web browser, NOT by you. Your web browser will have no difficulty working with this HTML. Note the inclusion of a link to a CSS file (more on this in a bit).

The `include_footer()` function produces HTML that terminates a web page, providing links (if provided as a dictionary). An empty dictionary switches off the inclusion of the linking HTML:

```
>>> include_footer({'Home': '/index.html', 'Select': '/cgi-bin/select.py'})
'<p>\n<a href="/index.html">Home</a>&nbsp;&nbsp;&nbsp;&nbsp;<a href="/cgi-bin/select.py">Select</a>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;</p>\n</body>\n</html>\n'
>>> include_footer({})
'<p>\n</p>\n</body>\n</html>\n'
```

With links included, and without.

The `start_form()` and `end_form()` functions bookend a HTML form, with the parameter (if supplied) adjusting the contents of the generated HTML:

```
>>> start_form("/cgi-bin/process-athlete.py")
'<form action="/cgi-bin/process-athlete.py" method="POST">'
>>> end_form()
'<p></p><input type="submit" value="Submit"></form>'
>>> end_form("Click to Confirm Your Order")
'<p></p><input type="submit" value="Click to Confirm Your Order"></form>'
```

The argument allows you to specify the name of the program on the server to send the form's data to.

HTML radio buttons are easy to create with the `radio_button()` function:

```
>>> for fab in ['John', 'Paul', 'George', 'Ringo']:
    radio_button(fab, fab)

'<input type="radio" name="John" value="John"> John<br />'
'<input type="radio" name="Paul" value="Paul"> Paul<br />'
'<input type="radio" name="George" value="George"> George<br />'
'<input type="radio" name="Ringo" value="Ringo"> Ringo<br />'
```

Which one is your favorite? Select from the list of radio buttons.

Unordered lists are a breeze with the `u_list()` function:

```
u_list(['Life of Brian', 'Holy Grail'])
'<ul><li>Life of Brian</li><li>Holy Grail</li></ul>'
```

Again, not too easy on your eye, but fine as far as your web browser is concerned.

The `header()` function lets you quickly format HTML headings at a selected level (with 2 as the default):

```
>>> header("Welcome to my home on the web")
'<h2>Welcome to my home on the web</h2>'
>>> header("This is a sub-sub-sub-sub heading", 5)
'<h5>This is a sub-sub-sub-sub heading</h5>'
```

Nothing too exciting here, but it works as expected. Same goes for here.

Last, but not least, the `para()` function encloses a chunk of text within HTML paragraph tags:

```
>>> para("Was it worth the wait? We hope it was...")
'<p>Was it worth the wait? We hope it was...</p>'
```

Control your code

Your **model** code is ready, and you have a good idea of how the **yate** module can help you with your **view** code. It's time to glue it all together with some **controller** code.

First things first: you need to arrange your webapp's directory structure to help keep things organized. To be honest, anything goes here, although by giving it a little thought, you can enhance your ability to extend your webapp over time. Here's one folder structure that *Head First Labs* recommends.



You can call your top-level folder anything you like.



As well as containing the subfolders, this folder contains your webapp's "index.html" file, your "favicon.ico" icon, style sheets, and anything else that doesn't fit neatly into one of the subfolders.

cgi-bin

Any code that you write for your webapp needs to reside in a specially named folder called "cgi-bin".

data

Let's keep the coach's data files in a separate folder by putting all of the TXT files in here.

images

If your webapp has any images files (JPGs, GIFs, PNGs, and so on), pop them into their own folder to help keep things organized.

templates

The templates that came with the "yate.py" download can go in here.

Do this!

Head on over to the *Head First Python* support website, download `webapp.zip`, and unpack it to your hard disk.

CGI lets your web server run programs

The Common Gateway Interface (CGI) is an Internet standard that allows for a web server to run a **server-side program**, known as a *CGI script*.

Typically, CGI scripts are placed inside a special folder called `cgi-bin`, so that the web server knows where to find them. On some operating systems (most notably UNIX-styled systems), CGI scripts must be set to *executable* before the web server can execute them when responding to a web request.

↑
More on this in
a little bit.



So...to run my webapp,
I need a web server with
CGI enabled.

I'm all fired up and
ready to go! I live to
serve-up HTML and
run CGIs...



All webapps need to run on web servers.

Practically every web server on the planet supports CGI. Whether you're running *Apache*, *IIS*, *nginx*, *Lighttpd*, or any of the others, they all support running CGI scripts written in Python.

But using one of these tools here is *overkill*. There's no way the coach is going to agree to download, unpack, install, configure, and manage one of these industry heavyweights.

As luck would have it, Python comes with its very own web server, included in the `http.server` library module. Check the contents of the `webapp.zip` download: it comes with a CGI-enabled web server called `simplehttpd.py`.

Import the HTTP
server and CGI
modules.

Specify a port.

Create a
HTTP server.

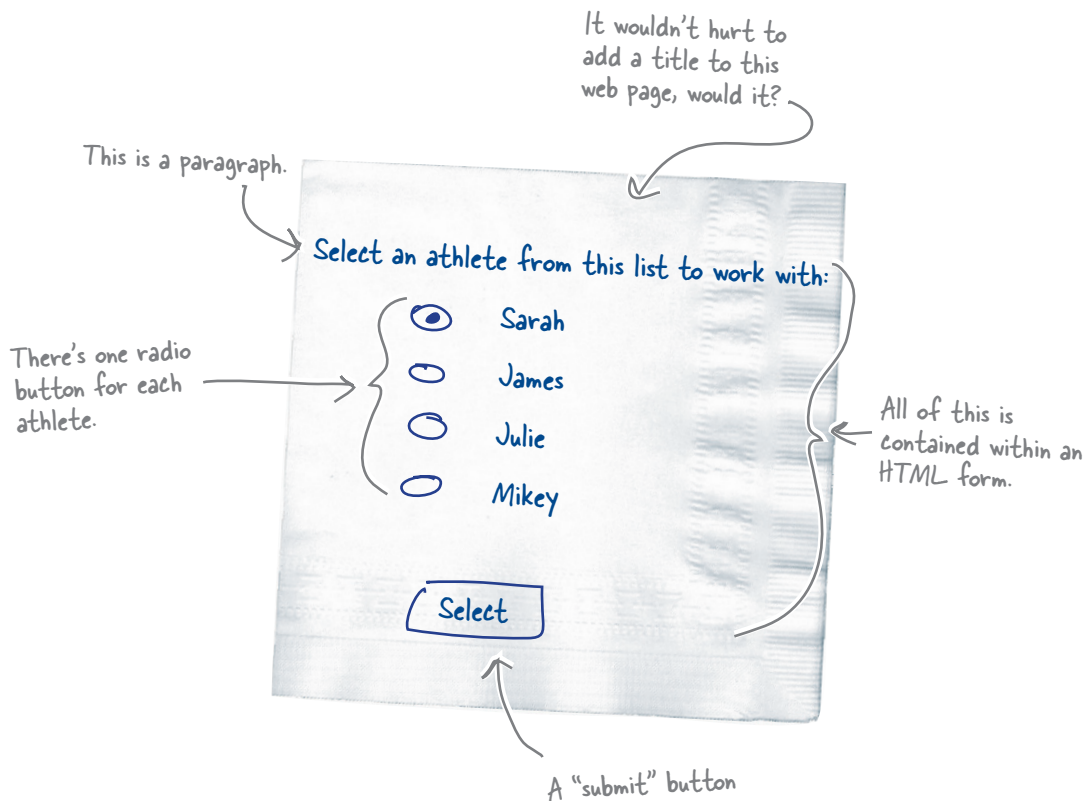
Display a friendly
message and start
your server.

Here are the five lines of code needed to
build a web server in Python.

```
from http.server import HTTPServer, CGIHTTPRequestHandler
port = 8080
httpd = HTTPServer(('', port), CGIHTTPRequestHandler)
{ print("Starting simple_httpd on port: " + str(httpd.server_port))
  httpd.serve_forever()
```

Display the list of athletes

Let's create a program called `generate_list.py` which, when executed by the web server, dynamically generates a HTML web page that looks something like this:



When your user selects an athlete by clicking on her radio button and clicking Select, a *new* web request is sent to the web server. This new web request contains data about which radio button was pressed, *as well as the name of a CGI script to send the form's data to.*

Recall that all of your CGI scripts need to reside in the `cgi-bin` folder on your web server. With this in mind, let's make sure your `generate_list.py` CGI script sends its data to another program called:

`cgi-bin/generate_timing_data.py`

Pool Puzzle



Your **job** is to take the code from the pool and place them into the blank lines in the CGI script. You may **not** use the same line of code more than once. Your **goal** is to make a CGI script that will generate a HTML page that matches the hand-drawn design from the previous page.

I've started things off for you.

```
import athletemodel
import yate
import glob
```

Import the modules that you need. You've already met "athletemodel" and "yate". The "glob" module lets you query your operating system for a list of file names.

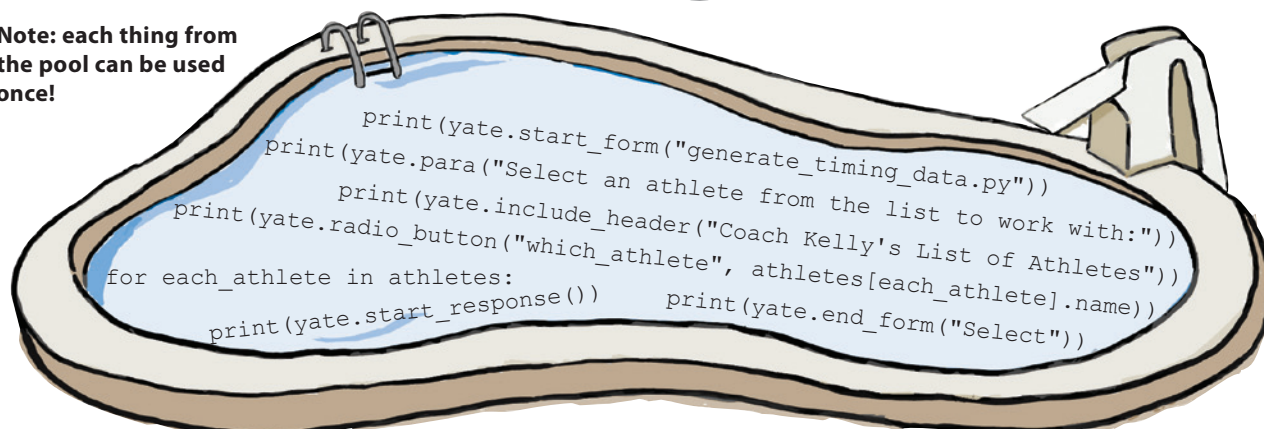
```
data_files = glob.glob("data/*.txt")
athletes = athletemodel.put_to_store(data_files)
```

Use your "put_to_store()" function to create a dictionary of athletes from the list of data files.

Let's add a link to the bottom of the generated HTML page that takes your user home.

```
print(yate.include_footer({"Home": "/index.html"}))
```

Note: each thing from the pool can be used once!



Pool Puzzle Solution



Your **job** was to take the code from the pool and place them into the blank lines in the CGI script. You were **not** to use the same line of code more than once. Your **goal** was to make a CGI script that generates a HTML page that matches the hand-drawn design.

```
import athletemodel
import yate
import glob
```

```
data_files = glob.glob("data/*.txt")
athletes = athletemodel.put_to_store(data_files)
```

```
.....
print(yate.start_response())
print(yate.include_header("Coach Kelly's List of Athletes"))
print(yate.start_form("generate_timing_data.py"))
print(yate.para("Select an athlete from the list to work with:"))
.....
```

```
.....
for each_athlete in athletes:
    print(yate.radio_button("which_athlete", athletes[each_athlete].name))
    print(yate.end_form("Select"))
.....
print(yate.include_footer({"Home": "/index.html"}))
```

Always start with a Content-type line.

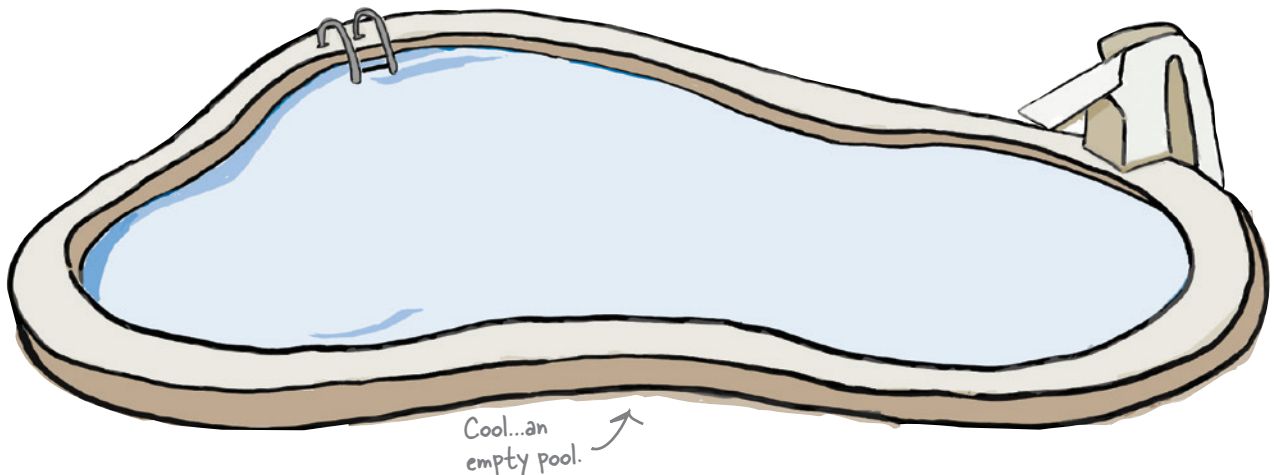
Start generating the form, providing the name of the server-side program to link to.

Generate a radio-button for each of your athletes.

Start generating the web page, providing an appropriate title.

A paragraph telling your user what to do

End the form generation with a custom "Submit" button.





Watch it!

What you need to do next depends on the operating system you're running your web server on.

If you are running on Windows, stop reading right now and proceed to the Test Drive. However, if you are running a Unix-based system (such as Linux, Mac OS X, or BSD) you need to do two things to prepare your CGI script for execution:

1. Set the **executable bit** for your CGI using the **chmod +x** command.
2. Add the following line of code to the very top of your program:

```
#!/usr/local/bin/python3
```

From your terminal window, type
`chmod +x generate_list.py`
 to set the executable bit. You
 need do this only once.



Test Drive

To test drive your CGI script, you need to have a web server up and running. The code to `simplehttpd.py` is included as part of the `webapp.zip` download. After you unpack the ZIP file, open a terminal window in the `webapp` folder and start your web server:

File Edit Window Help WebServerOnWindows

```
c:\Python31\python.exe simplehttpd.py
Starting simple_httpd on port: 8080
```

Use this command
 on Windows-based
 systems.

File Edit Window Help WebServerOnUNIX

```
$ python3 simple_httpd.py
Starting simple_httpd on port: 8080
```

Use this command
 on Unix-based
 systems.



— Test Drive, continued

With your web server running, let's load up Coach Kelly's home page and get things going. You've started your web server running on port 8080 on your computer, so you need to use the following web address in your web browser: `http://localhost:8080`.



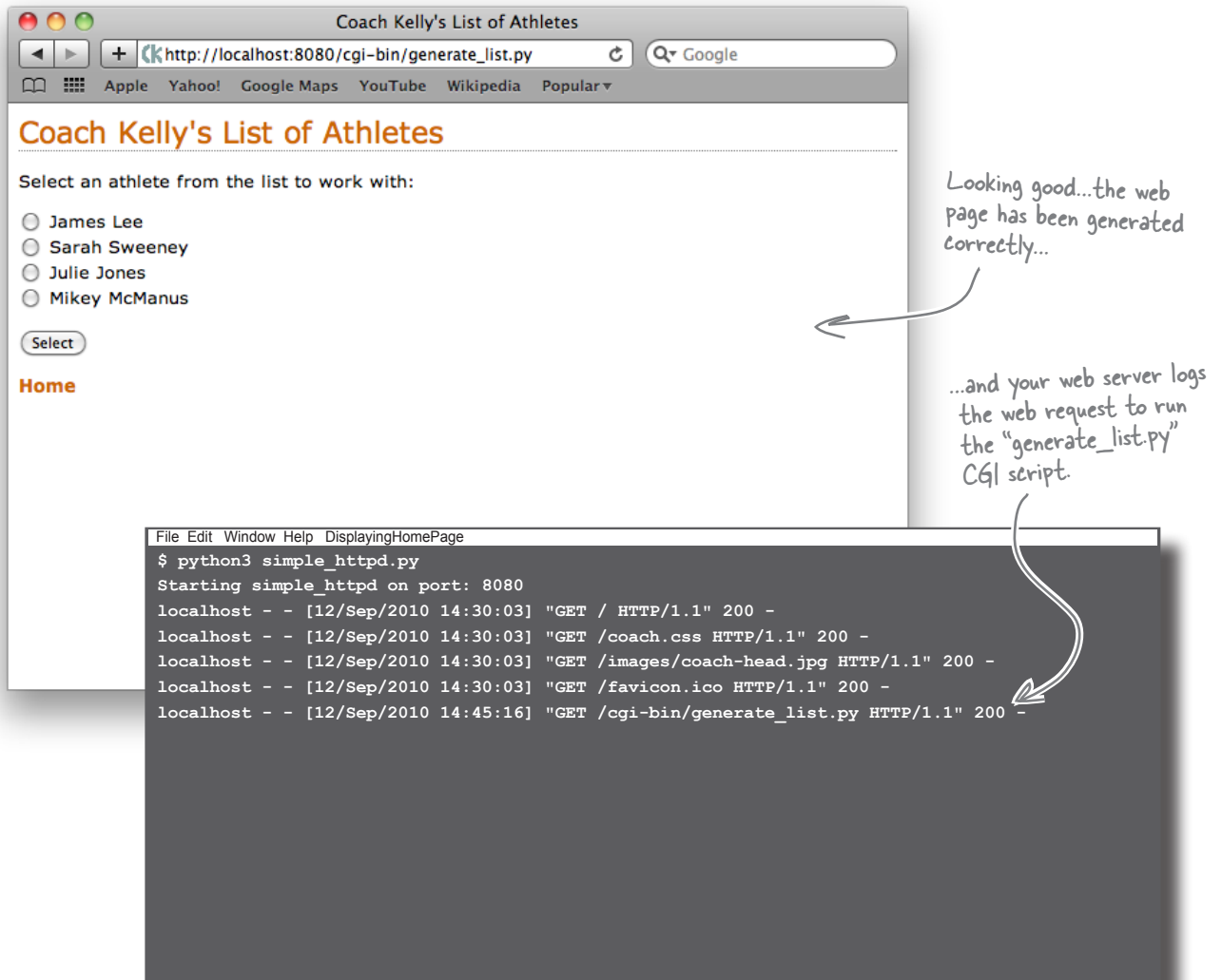
The coach's home page appears in your browser. It's called "index.html" and it is included in the "webapp.zip" download.

...and your web server springs into life, logging (to the screen) any and all web requests that it processes.

The "timing data" hyperlink is waiting for you to click it.

```
File Edit Window Help DisplayingHomePage
$ python3 simple_httpd.py
Starting simple_httpd on port: 8080
localhost - - [12/Sep/2010 14:30:03] "GET / HTTP/1.1" 200 -
localhost - - [12/Sep/2010 14:30:03] "GET /coach.css HTTP/1.1" 200 -
localhost - - [12/Sep/2010 14:30:03] "GET /images/coach-head.jpg HTTP/1.1" 200 -
localhost - - [12/Sep/2010 14:30:03] "GET /favicon.ico HTTP/1.1" 200 -
```

Sure enough, clicking on the home page's link runs the `generate_list.py` program on the web server, which displays Coach Kelly's athletes as a list of radio buttons.

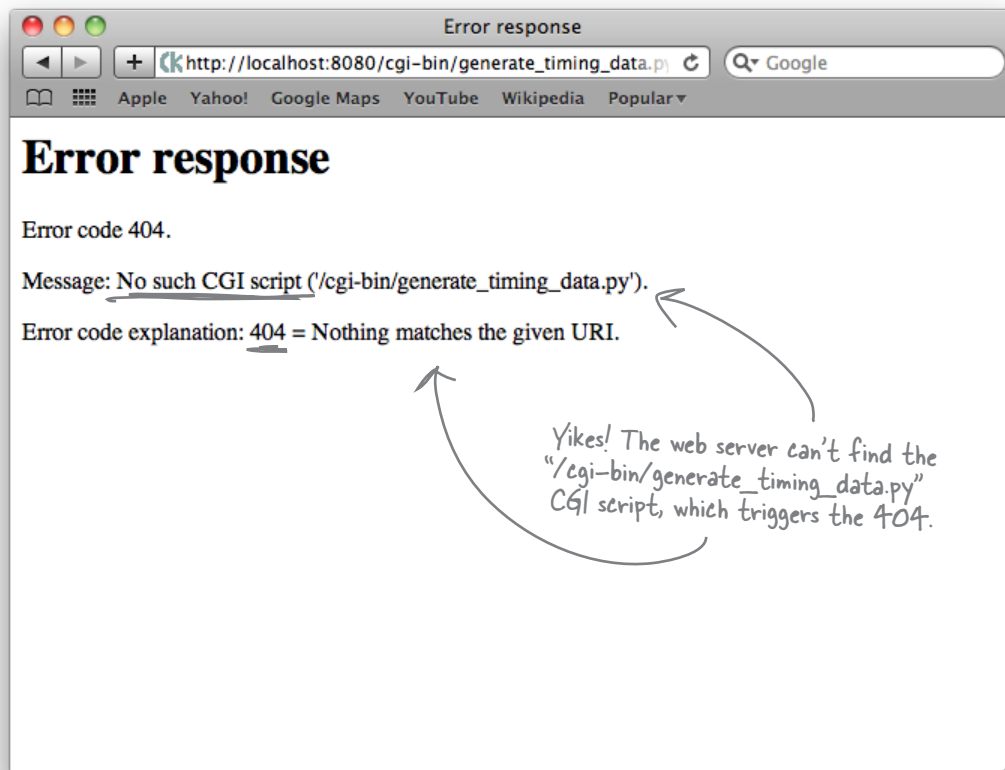


You can click the Home hyperlink to return to the coach's home page, or select an athlete from the list (by clicking on their radio-button), before pressing the Select button to continue.

Select an athlete and press Select. What happens?

The dreaded 404 error!

Whoops! Your web server has responded with a “404” error code, which is its way of telling you that something was wrong with *your* request. The web server is in fact telling you that it can’t locate the resource that your web browser requested, so it’s telling you that *you* made a mistake:



Check the web server’s console window to confirm that your attempt to post your form’s data to `generate_timing_data.py` resulted in failure.

Which isn’t really that surprising seeing as *you have yet to write that code!* So...things aren’t as bad as they first appear. The “404” error is *exactly* what you would expect to be displayed in this situation, so your `generate_list.py` CGI is working fine. What’s needed is the code to the *other* CGI script.

If you create the required CGI script, you’ll be back on track.

Fireside Chats



Tonight's talk: **To be CGI or not to be CGI, that is the question.**

A Python Program:

Listen: you're really not all that different than me; you just work on a web server, whereas I can work *anywhere*.

Special?!? But you only work on the Web, nowhere else. How's that "special"?

Nonsense! The truth is that you work *only* on the Web and break pretty quickly when used elsewhere. You don't even have control over your own I/O.

Like [sniggers] generating text in the form of HTML? That's really taxing...

Oh, get over yourself! You're a regular program, just like me. I can generate HTML, too, I just choose not to.

I guess so...

Ummmm...I guess so.

A Python CGI Script:

Yes. I like to think of myself as *special*.

Because all the cool stuff works on the Web these days and I'm designed, optimized, tailored, and engineered for the Web. Because the Web's a cool place, it follows that I must be cool, too. See: *special*.

I don't need control over my input and output. I have a friendly web server to take care of that for me. My input *comes from* the web server and my output *goes to* the web server. This arrangement allows me to concentrate on the important stuff.

Smirk all you want; HTML makes the World Wide Web go around and I'm a master at generating it *dynamically, on demand, and as needed*. Without me, the Web would be a pretty static place.

And if you did generate HTML, you'd want it displayed somewhere...like in a browser?

And to do that you'd need to rely on the services of a friendly web server, right?

Which would make you a CGI script. So, you'd be *special*, too. Q.E.D.

Create another CGI script

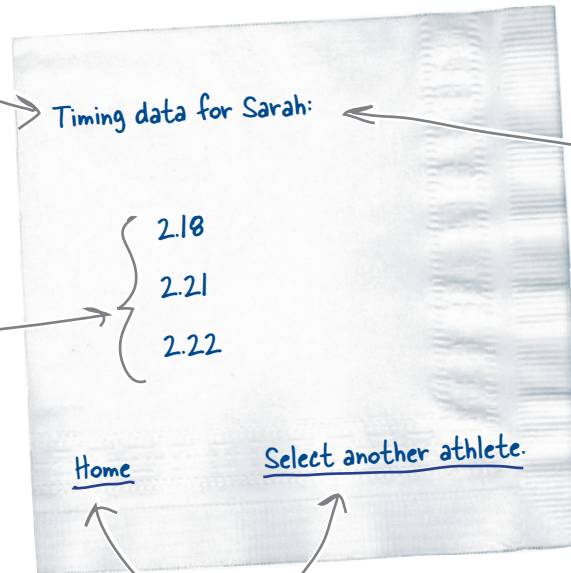
Let's take a moment to recall what is required from the `generate_timing_data.py` CGI script. Based on your hand-drawn sketch from earlier, your need to generate a new HTML page that contains the top three times for the selected athlete:

This looks like a HTML heading for the page, right?

It's probably a good idea to add a title to this page.

This might be best rendered as an unordered HTML list.

Let's include the athlete's full name and DOB here.



Two hyperlinks: one jumps to the home page, whereas the other returns to the previous selection page.

But how do you know which athlete is selected?

When you click on a radio-button and then press the Select button, a new web request is sent to the server. The web request identifies the CGI script to execute (in this case, that's `generate_timing_data.py`), together with the form's data. The web server arranges to send the form's data to your CGI script *as its input*. Within your code, you can access the form data using Python's `cgi` module, which is part of the standard library:

Import the "cgi" library.

```
import cgi
form_data = cgi.FieldStorage()
athlete_name = form_data['which_athlete'].value
```

Grab all of the form data and put it in a dictionary.

Access a named piece of data from the form's data.



Sharpen your pencil

Write the code
to your new CGI
script here.

It's time to exercise your newly acquired web-coding chops. Grab your pencil and write the code for the `generate_timing_data.py` CGI script. It's not too different from the `generate_list.py` code, so you should be able to reuse a lot of your existing code.



Sharpen your pencil Solution

It's time to exercise your newly acquired web-coding chops. You were to grab your pencil and write the code for the `generate_timing_data.py` CGI script. It's not too different from the `generate_list.py` code, so you should be able to reuse a lot of your existing code.

```
#!/usr/local/bin/python3
```

← This line is needed on Unix-based systems only.

```
import cgi
import athletemodel
import yate
```

Import the libraries and modules you intend to use.

```
athletes = athletemodel.get_from_store()
```

Get the data from the model.

```
form_data = cgi.FieldStorage()
athlete_name = form_data['which_athlete'].value
```

Which athlete's data are you working with?

```
print(yate.start_response())
print(yate.include_header("Coach Kelly's Timing Data"))
print(yate.header("Athlete: " + athlete_name + ", DOB: " +
athletes[athlete_name].dob + "."))
```

Nothing new here or here.

← Grab the athlete's name and DOB.

```
print(yate.para("The top times for this athlete are:"))
print(yate.u_list(athletes[athlete_name].top3()))
```

← Turn the top three list into an unordered HTML list.

```
print(yate.include_footer({"Home": "/index.html",
"Select another athlete": "generate_list.py"}))
```

The bottom of this web page has two links.

← A link back to the previous CGI script.



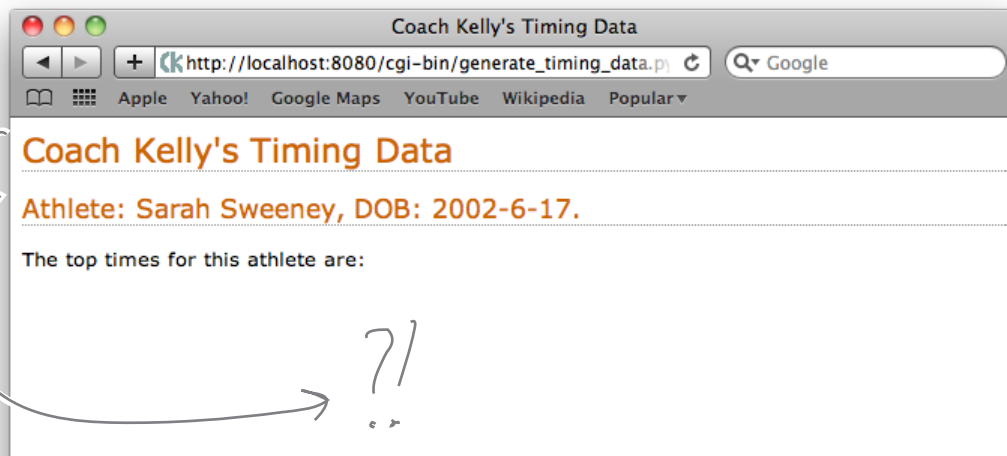
Test Drive

Note: If you are on a Unix-based system, don't forget to add "chmod +x generate_timing_data.py" to set the executable bit.

Your web server should still be running from earlier. If it isn't, start it again. In your web browser, return to the coach's home page, then select the hyperlink to display the list of athletes, select Sarah, and then press the button.

This all looks OK.

Ah, phooey! Something's not quite right here. Where's Sarah's top three times?



Does the web server's logging information tell you anything?

```
File Edit Window Help HoustonWeHaveAProblem
$ python3 simple_httpd.py
Starting simple_httpd on port: 8080
localhost - - [12/Sep/2010 14:30:03] "GET / HTTP/1.1" 200 -
localhost - - [12/Sep/2010 14:30:03] "GET /coach.css HTTP/1.1" 200 -
localhost - - [12/Sep/2010 14:30:03] "GET /images/coach-head.jpg HTTP/1.1" 200 -
localhost - - [12/Sep/2010 14:30:03] "GET /favicon.ico HTTP/1.1" 200 -
localhost - - [12/Sep/2010 14:45:16] "GET /cgi-bin/generate_list.py HTTP/1.1" 200 -
localhost - - [12/Sep/2010 16:12:27] "GET /cgi-bin/generate_list.py HTTP/1.1" 200 -
localhost - - [12/Sep/2010 16:12:29] "POST /cgi-bin/generate_timing_data.py HTTP/1.1" 200 -
Traceback (most recent call last):
  File "/Users/barryp/HeadFirstPython/chapter7/cgi-bin/generate_timing_data.py", line 21, in <module>
    print(yate.u_list(athletes[athlete_name].top3()))
TypeError: 'list' object is not callable
localhost - - [12/Sep/2010 16:12:29] CGI script exit status 0x100
```

Your CGI has suffered from a `TypeError` exception, but other than looking at the web server's logging screen, it's not clear *on the web browser screen* that anything has gone wrong.



BRAIN BARBELL

What do you think is the problem here? Take a moment to study the error message before flipping the page.

Enable CGI tracking to help with errors

The CGI standard dictates that any output generated by a server-side program (your CGI script) should be captured by the web server and sent to the waiting web browser. Specifically, anything sent to `STDOUT` (standard output) is captured.

When your CGI script raises an exception, Python arranges for the error message to display on `STDERR` (standard error). The CGI mechanism is programmed to ignore this output because all it wants is the CGI script's standard output.



This behavior is fine when the webapp is deployed, but *not* when it's being developed. Wouldn't it be useful to see the details of the exception in the browser window, as opposed to constantly having to jump to the web server's logging screen?

Well...guess what? Python's standard library comes with a CGI tracking module (called `cgitb`) that, when enabled, arranges for detailed error messages to appear in your web browser. These messages can help you work out where your CGI has gone wrong. When you've fixed the error and your CGI is working well, simply switch off CGI tracking:

```
import cgitb
cgitb.enable()
```

Add these two lines near the start of your CGI scripts to enable Python's CGI tracking technology.



Test Drive

Add the two CGI tracking lines of code near the top of your `generate_timing_data.py` CGI script. Press the Back button on your web browser and press the Select button again. Let's see what happens this time.

Wow! Look at all of this detail.

Coach Kelly's Timing Data

Athlete: Sarah Sweeney, DOB: 2002-6-17.

The top times for this athlete are:

TypeError /Library/Frameworks/Python.framework/Versions/3.1/Resources/Python.app/Contents/MacOS/Python
Sun Sep 12 20:13:30

A problem occurred in a Python script. Here is the sequence of function calls leading up to the error, in the order they occurred.

```

/Users/barryp/HeadFirstPython/chapter7/cgi-bin/generate_timing_data.py in ()
19         athletes[athlete_name].dob + ".")
20 print(yate.para("The top times for this athlete are:"))
=> 21 print(yate.u_list(athletes[athlete_name].top3()))
22 print(yate.include_footer({"Home": "/index.html",
23                             "Select another athlete": "generate_list.py"}))

builtin print = <built-in function print>, yate = <module 'yate' from '/Users/barryp/HeadFirstPython/chapter7/cgi-
bin/yate.py'>, yate.u_list = <function u_list>, athletes = {'James Lee': ['2-34', '2:21', '2:34', '2:45', '3:01', '2:01',
'2:01', '3:10', '2-22', '2-01', '2:01', '2:16'], 'Julie Jones': ['2:59', '2:11', '2:11', '2:23', '3-10', '2-23', '3:10', '3:21', '3-21',
'3:01', '3:02', '2:59'], 'Mikey McManus': ['2:22', '3:01', '3:01', '3:02', '3:02', '3:02', '3:22', '2:49', '2:38', '2:40', '2:22', '2
31'], 'Sarah Sweeney': ['2:58', '2:58', '2:39', '2-25', '2-55', '2:54', '2:18', '2:55', '2:55', '2:22', '2-21', '2:22']},
athlete_name = 'Sarah Sweeney', 1.top3 undefined
  
```

TypeError: 'list' object is not callable
args = ("list' object is not callable",)
with_traceback = <built-in method with_traceback of TypeError object>

What's this? `top3()` is undefined?!?

Notice that the CGI tracking module tries to identify *exactly* where the problem with your code lies.

A small change can make all the difference

The CGI tracking output indicates an error with the use of the `top3()` method from the `AthleteList` code.

A quick review of the code to the `AthleteList` class uncovers the source of the error: the `top3()` method has been redesignated as a **class property**.

`@property`

`def top3(self):`

`return(sorted(set([self.sanitize(t) for t in self]))[0:3])`

This decorator allows you to access the data returned by "top3()" as if it were a class attribute.

The use of the `@property` decorator allows the `top3()` method to appear like an attribute to users of the class. So, instead of calling the `top3()` method like this:

`print(yate.u_list(athletes[athlete_name].top3()))`

A method call always needs the parentheses...

Treat the `top3()` method as if it was another class attribute, and call it like this:

`print(yate.u_list(athletes[athlete_name].top3))`

It's a small change, but it's an important one

When a change is made to the way a class is used, you need to be careful to consider what impact the change has on existing programs, both yours and those written by others.

At the moment, you are the only one using the `AthleteList` class, so it's not a big deal to fix this. But imagine if thousands of programmers were using and relying on your code...

...unless the method is declared to be an "`@property`", in which case parentheses are NOT required.

Let's fix your CGI script and try again.

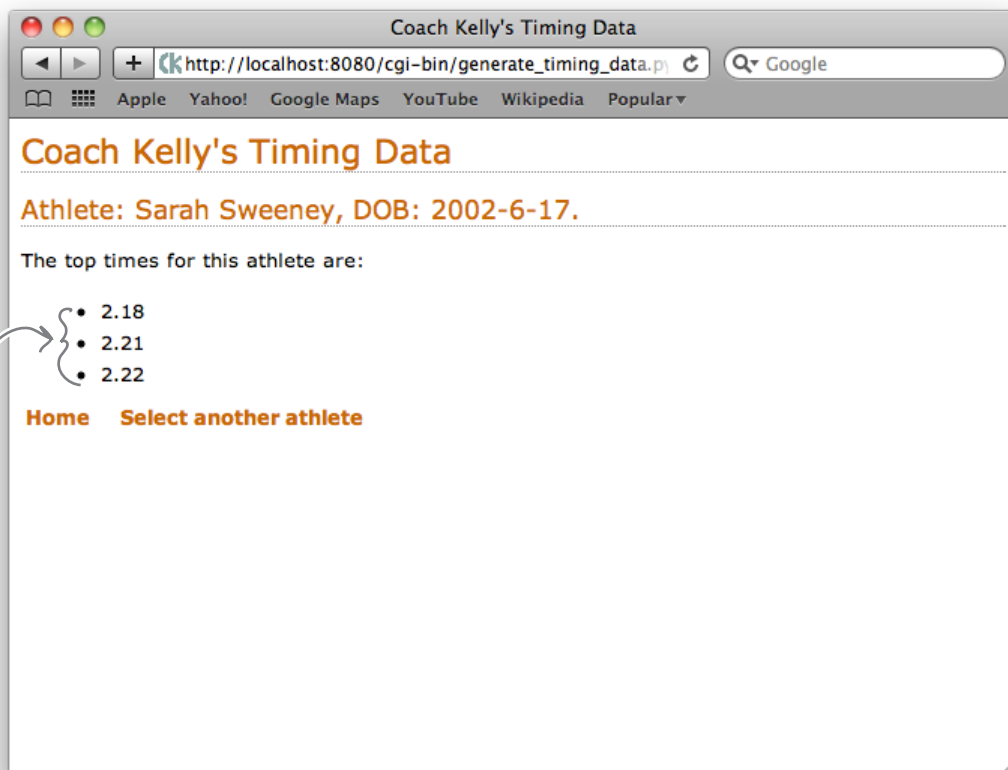


Test Drive

Make the small edit to your code to remove the brackets from the call to the `top3()` method, press your web browser's Back button, and press the Select button one last time.

Whoa hoo! This time the selected athlete's data is displayed on screen. Nice, eh?

Now that you've solved that problem, be sure to switch off CGI tracking.



there are no Dumb Questions

Q: What happens if the coach recruits new athletes?

A: All Coach Kelly needs do is create a new text file similar to the others, and your webapp handles the rest by dynamically including the new athlete the next time your webapp runs, which occurs when someone clicks on the home page's "timing data" hyperlink.

Q: Shouldn't the server's data be in a database as opposed to a pickle? Surely that would be better, right?

A: In this case, it's probably overkill to use a database, but it might be worth considering sometime in the future.

Your webapp's a hit!



By moving your program to the Web, you've made it a no-brainer for Coach Kelly to share his data with not only his athletes, but with *anyone* else that needs to access his data.

By conforming to the **MVC** pattern and using **CGI**, you've built a webapp in such a way that it's easy to extend as new requirements are identified.

Congratulations! You're a web developer.



Your Python Toolbox

You've got Chapter 7 under your belt and you've added some key Python techniques to your toolbox.

Python Lingo

- “@property” – a decorator that lets you arrange for a class method to appear as if it is a class attribute.

Web Lingo

- “webapp” – a program that runs on the Web.
- “web request” – sent from the web browser to the web server.
- “web response” – sent from the web server to the web browser in response to a web request.
- “CGI” – the Common Gateway Interface, which allows a web server to run a server-side program.
- “CGI script” – another name for a server-side program.

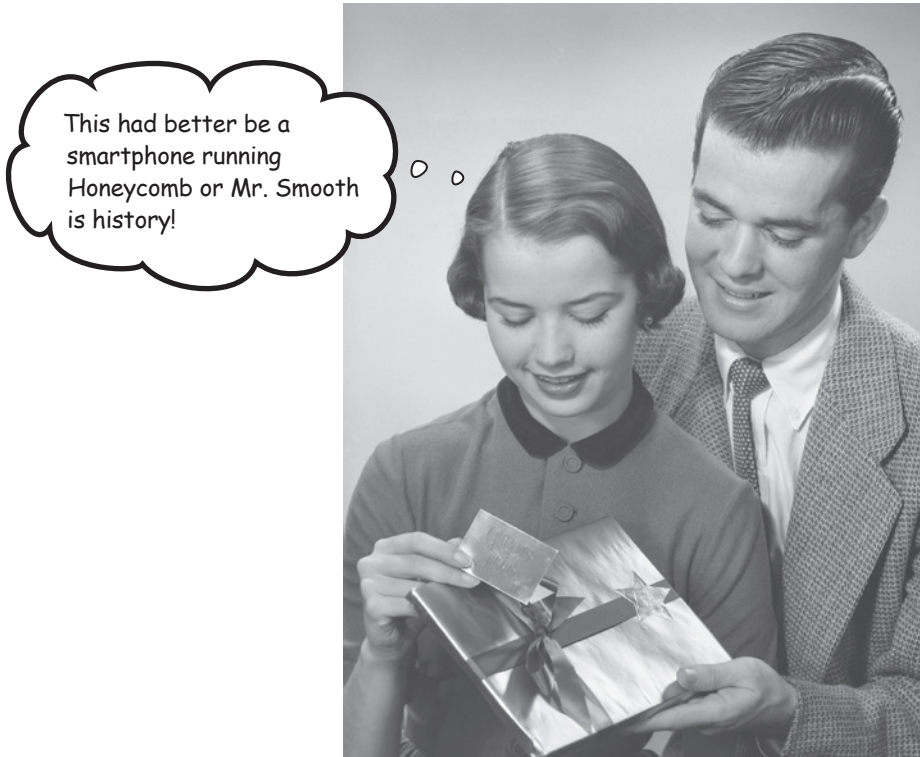


BULLET POINTS

- The *Model-View-Controller* pattern lets you design and build a webapp in a maintainable way.
- The **model** stores your webapp's data.
- The **view** displays your webapp's user interface.
- The **controller** glues everything together with programmed logic.
- The standard library `string` module includes a class called `Template`, which supports simple string substitutions.
- The standard library `http.server` module can be used to build a simple web server in Python.
- The standard library `cgi` module provides support for writing CGI scripts.
- The standard library `glob` module is great for working with lists of filenames.
- Set the **executable** bit with the `chmod +x` command on *Linux* and *Mac OS X*.
- The standard library `cgitb` module, when enabled, lets you see CGI coding errors within your browser.
- Use `cgitb.enable()` to switch on CGI tracking in your CGI code.
- Use `cgi.FieldStorage()` to access data sent to a web server as part of a web request; the data arrives as a Python dictionary.

8 mobile app development

Small devices



Putting your data on the Web opens up all types of possibilities.

Not only can anyone from anywhere interact with your webapp, but they are increasingly doing so from a collection of diverse computing devices: PCs, laptops, tablets, palmtops, and even mobile phones. And it's not just humans interacting with your webapp that you have to support and worry about: *bots* are small programs that can automate web interactions and typically want your data, not your human-friendly HTML. In this chapter, you exploit Python on Coach Kelly's mobile phone to write an app that interacts with your webapp's data.

The world is getting smaller

Coach Kelly is continuing to use his webapp every day, but he's having a problem with his new smartphone.



There's more than just desktop computers out there.

Who knew that your users would try to interact with your webapp using something other than a desktop computer or laptop?

It's a diverse computing environment out there.

Coach Kelly is on Android

The coach has a lovely new smartphone that's running Google's Android operating system. Sure enough, when you check it out, the webapp is way too small and not much use on the coach's three-inch screen:



Does anyone have a magnifying glass?

And don't go telling me to do all that two-fingered zoom and double-tapping thing. That just drives me crazy!



Obviously, the coach needs to access his data and run his webapp on his phone...but what's the *best way* to do this if not through the phone's browser?



Sharpen your pencil

Open your web browser on your desktop computer (or phone) and enter "Python for Android" into your favorite search engine. Make a note in the space below of the most promising site from your search results:

.....

.....

Sharpen your pencil Solution

Is this the one
you found?

<http://code.google.com/p/android-scripting> (the home of the SL4A project.)

You were to open your web browser on your desktop computer (or phone) and enter “Python for Android” into your favorite search engine. You were then to make a note in the space below of the most promising site from your search results:

Run Python on the coach’s smartphone

A quick search of the Web uncovers a pleasant surprise: *Python runs on Android*.

At least a *version* of Python runs on Android. A project called *Scripting Layer for Android (SL4A)* provides technology to let you run Python on any Android device. But there’s a catch.



Ummmm...I just checked
the SL4A website, and
it looks like it supports
Python 2.6.2, not Python 3.
Phooey!



Yes. SL4A ships with Python 2, not 3.

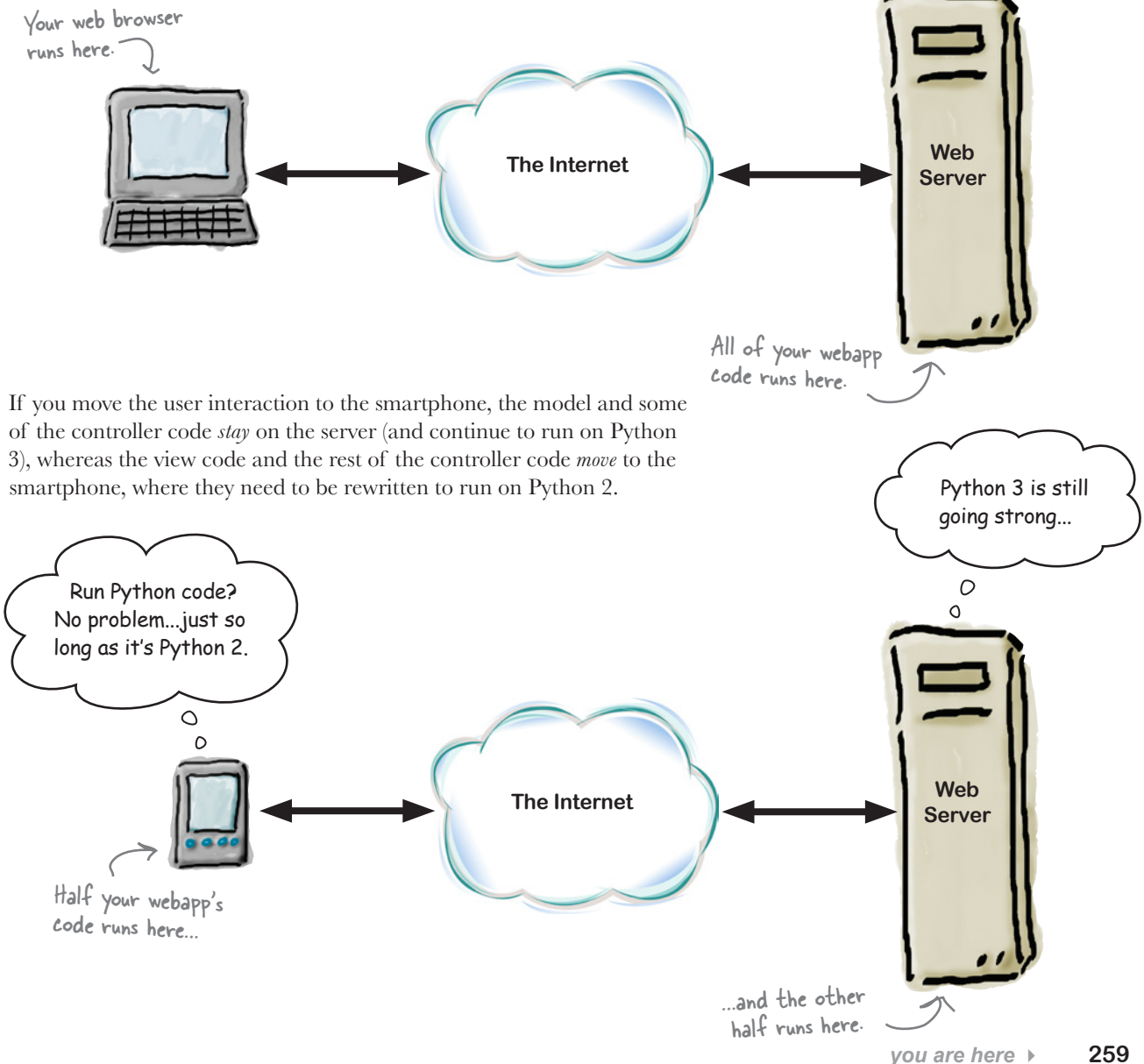
Python 3, this book’s preference, is the best version of Python yet, but it achieves its greatness at the cost of a lack of backward compatibility. There’s some stuff in 3 that will never work in 2 and vice versa.

Is this fact alone a *show-stopper*?

Don't worry about Python 2

The fact that Python 2 is available for Android and you've learned Python 3 in this book is nothing to lose sleep over. *Python 2 is still Python*, and the differences between Python 2 and Python 3 are easy to manage.

Think about your webapp for a minute. Right now, the model, view, and controller code resides on the web server, which is running Python 3.



Set up your development environment

Understandably, the coach won't let you have his phone to work on until you have something that works. Thankfully, *Google* provides a cross-platform **Android emulator** that lets you develop for the phone as needed, even though you don't own any hardware.

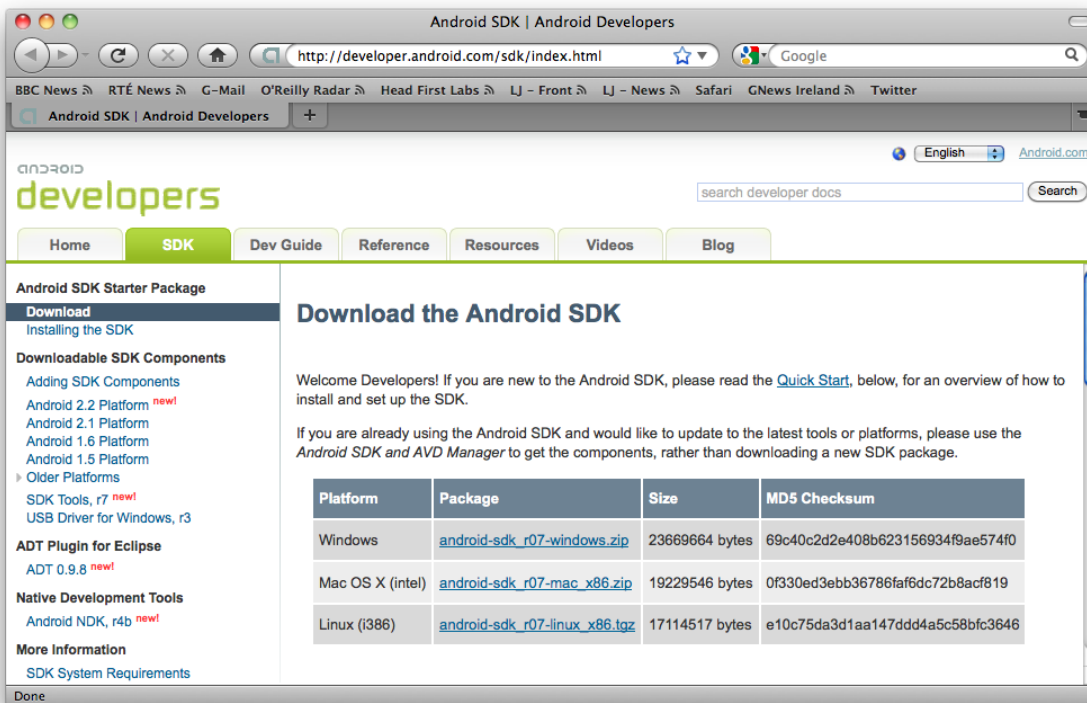
Download the Software Development Kit (SDK)

Let's get started developing for Android. Visit this website and download the SDK for your computer and operating system:

<http://developer.android.com/sdk/index.html>



Follow along with these instructions to ensure you have your Android development environment correctly set up on your computer.



The Android SDK website



Note: This is how the Android SDK download page looks at the time of this writing. It might look a little different for you. No worries: just download the latest version of the SDK.

Despite what this website might look like it's telling you, you do **not** need to install *Eclipse* to run the Android emulator. However, you do need to have a *Java Runtime Environment* installed. If you are unsure about this, don't worry: the Android emulator will advise your best course of action if it spots that Java is missing.

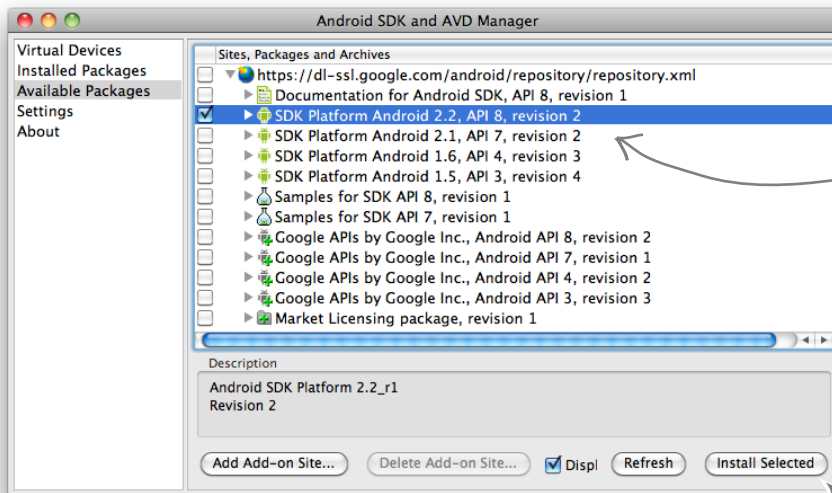
Configure the SDK and emulator

You need to do *two things* to configure the SDK and emulator: add an *Android Platform* and create an *Android Virtual Device* (known as an *AVD*).

Add an Android platform

The coach is running **Android 2.2** on his phone, so let's add a 2.2 platform to mimic this setup. Open up the Android SDK and AVD Manager tool, select Available Packages, and pick 2.2 for installation.

The Android download contains a folder called "tools". Run the "android" program within this folder.

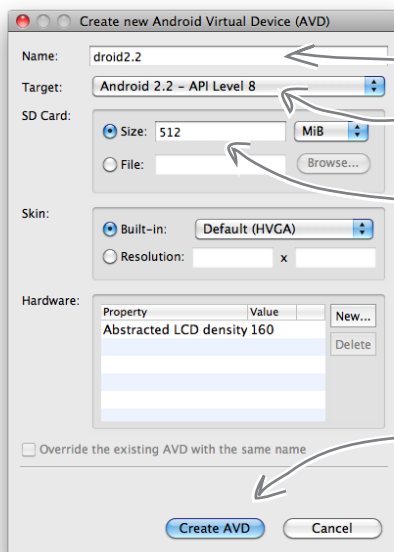


This is the only version of the SDK that you need.

This might take a minute or two, depending on the speed of your network connection.

Create a new Android Virtual Device (AVD)

With the 2.2 platform downloaded and installed, create a new Android Virtual Device.



Give your AVD a name, and select a target.

Set the size of the virtual SD card: 512 is more than enough.

Click on "Create AVD".

Your AVD is a simulated Android phone.

Install and configure Android Scripting

With the emulator ready, use the AVD Manager to start your 2.2 device. Click on the emulator's browser (the little globe), surf to this web address:

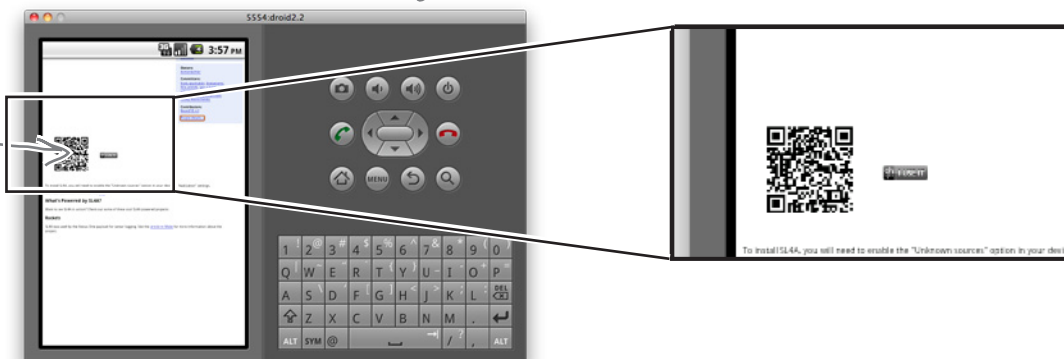
<http://code.google.com/p/android-scripting>

and tap on the “boxed” bar code near the bottom of the page:

These instructions work on a “real” phone, too. Just be sure to enable “Unknown sources” to allow for non-Market application downloads.

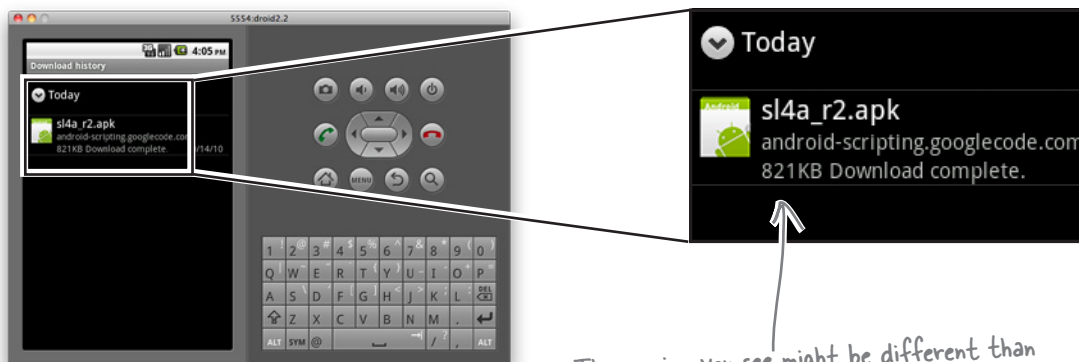
Don't worry if it takes your emulator a minute or two to start. The emulator is slower than the actual phone...

On the emulator, tap on the “boxed” bar code to start the SL4A download.



When the download completes, select the emulator's Menu button → More → Downloads, and then tap on the `sl4a_r2.apk` file to install the SL4A package on the emulator. When the install completes, tap Done.

The version available to you might be different, but don't worry: download the latest release.



The version you see might be different than this. Don't worry; yours is the most recent.

Add Python to your SL4A installation

Return to the emulator's web browser, double-tap on the screen to zoom in, and select the Downloads tab. Double-tap again and tap the following link:

python_for_android_r1.apk

Tap the download link, and tap on the package name to download it. Select Menu → More → Downloads, and tap on the newly downloaded package.

Again, the version you see might be different than this. Select the most recent file.

The sequence of screenshots illustrates the following steps:

- Downloads Tab:** A list of files is shown, including `sl4a_r2.apk`, `rhino_for_android_r1.apk`, `python_for_android_r1.apk`, and `perl_for_android_r1.apk`. The `python_for_android_r1.apk` file is highlighted.
- Download Progress:** The `python_for_android_r1.apk` file is shown with a progress bar and a SHA1 checksum: `b213ce6f601625e07468fc0a48`. A note says: "Use the SHA1 checksum shown to verify file integrity."
- Download History:** The `python_for_android_r1.apk` file is listed as "Download complete" with a size of 30.97KB.
- Downloads Tab (Again):** The `python_for_android_r1.apk` file is highlighted again, showing its size as 821KB.

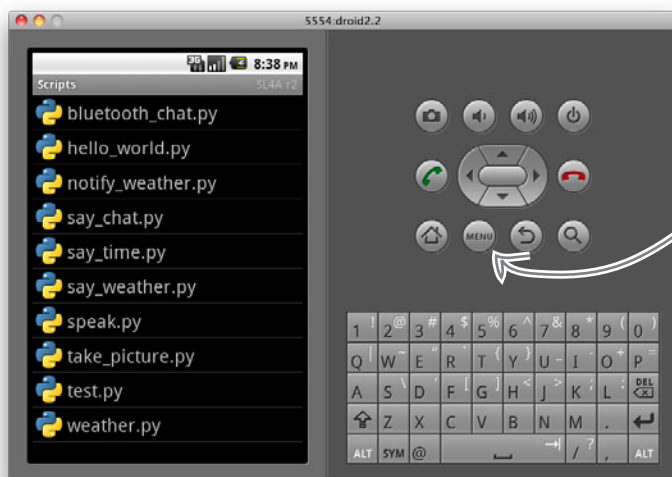
The Python for Android app runs. When you are ready, tap Open → Install to complete the installation. This downloads, extracts, and installs the Python support files for Android, which can take a few minutes to complete. When it does, Python 2.6.2 and Python for Android are installed on your emulator and ready for action.

This last bit is really important.

Let's confirm everything is working with a quick test.

Test Python on Android

Return to your emulator's main screen and find an app called *SL4A* added to your list of app icons. Tap this app to display the list of Python scripts preinstalled with Python for Android. Simply tap on any script name to execute it:



The "menu" button.



Watch it!

Be sure to set the SL4A rotation mode to automatic.

Your screen might switch to landscape by default the first time you run a script. To fix this, choose Menu → Preferences, scroll down to **Rotation mode**, and set its value to **Automatic**.

Take your Android emulator for a spin

Here's a four-line Python script that you can create to test your installation. Let's call this script `mydroidtest.py`:

Import the "android" library and create a new app object instance.

Create an appropriate message and display it on screen.

```
import android
app = android.Android()
msg = "Hello from Head First Python on Android"
app.makeToast(msg)
```

To transfer your script to the emulator, you need to copy it to the emulator's virtual SD card. Another program within the `tools` folder called `adb` helps with this:

Issue this command at your terminal window to transfer your script to the emulator.

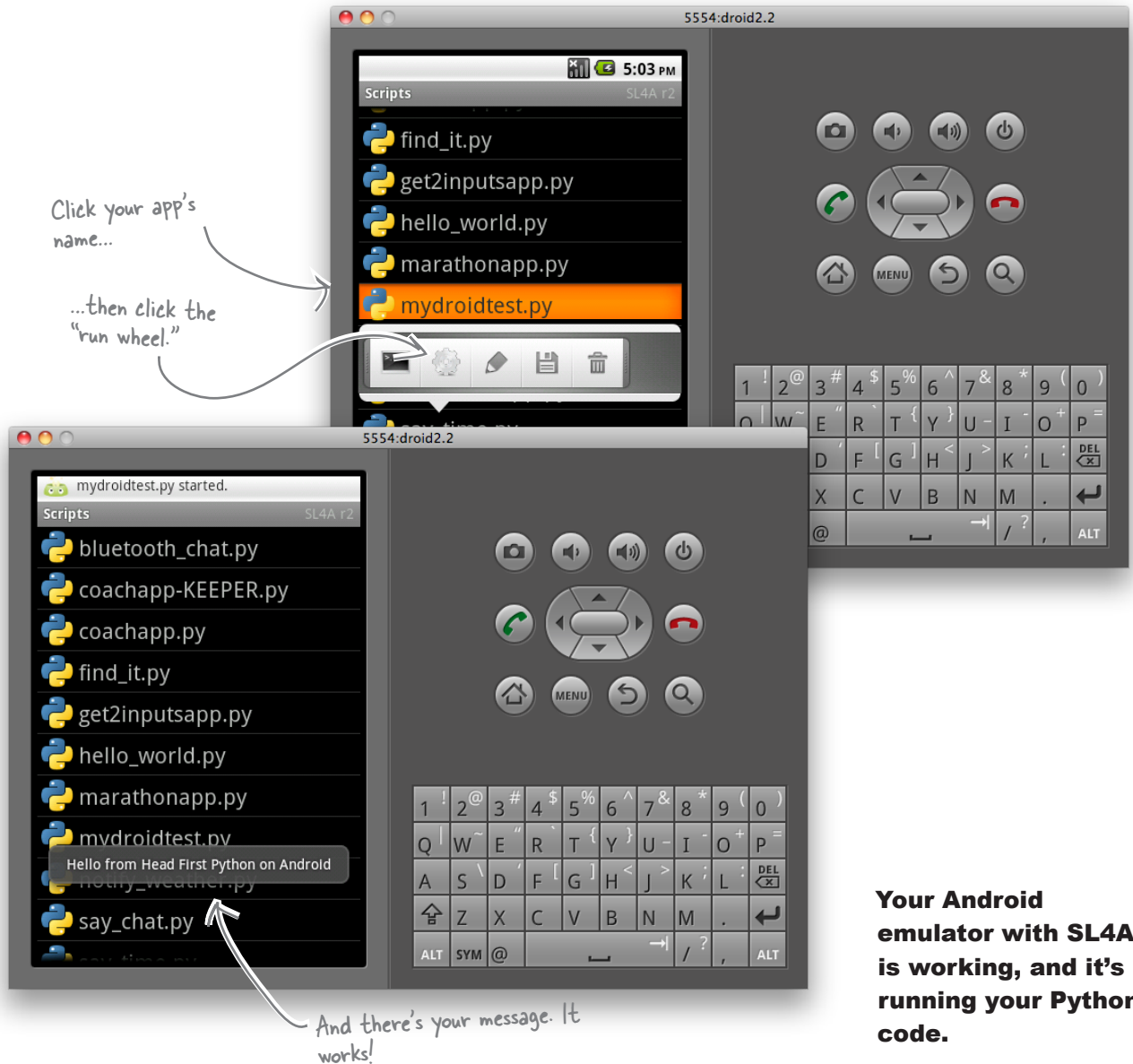
```
tools/adb push mydroidtest.py /sdcard/sl4a/scripts
```

Your script should now appear on the list of scripts available to SL4A.



Test Drive

Let's confirm that your Android setup is working. With the SL4A app open, simply tap on your script's name to run it, and then click the *run wheel* from the menu.



Define your app's requirements

Let's think a little bit about what your Android app needs to do.



Frank: Well...first off, the view code no longer has to generate HTML, so that makes things interesting.

Jill: In fact, you need the web server only to supply your data on request, not all that generated HTML.

Joe: Ah ha! I've solved it. Just send the pickle with all the data from the server to the Android phone. It can't be all that hard, can it?

Jill: Sorry, guys, that'll cause problems. The pickle format used by Python 3 is *incompatible* with Python 2. You'll certainly be able to send the pickle to the phone, but the phone's Python won't be able to work with the data in the pickle.

Frank: Darn...what are our options, then? Plain data?

Joe: Hey, good idea: just send the data as one big string and parse it on the phone. Sounds like a workable solution, right?

Jill: No, that's a potential disaster, because you never know in what format that stringed data will arrive. You need a *data interchange format*, something like XML or JSON.

Frank: Hmm...I've heard XML is a hound to work with...and it's probably overkill for this simple app. What's the deal with JSON?

Joe: Yes, of course, I keep hearing about JSON. I think they use it in lots of different places on the Web, especially with AJAX.

Frank: Oh, dear...pickle, XML, JSON, and now AJAX...I think my brain might just explode here.

Jill: Never worry, you only need to know JSON. In fact, you don't even need to worry about understanding JSON at all; you just need to know how to use it. And, guess what? JSON comes standard with Python 2 and with Python 3...and the format is compatible. So, we can use JSON on the web server and on the phone.

Frank & Joe: Bonus! That's the type of technology we like!



JSON Exposed

This week's interview:
The Data Interchange Lowdown

Head First: Hello, JSON. Thanks for agreeing to talk to us today.

JSON: No problem. Always willing to play my part in whatever way I can.

Head First: And what *is* that, exactly?

JSON: Oh, I'm just one of the most widely used data interchange formats on the Web. When you need to transfer data over the Internet, you can rely on me. And, of course, you'll find me *everywhere*.

Head First: Why's that?

JSON: Well...it's really to do with my name. The "JS" in JSON stands for "JavaScript" and the "ON" stands for "Object Notation." See?

Head First: Uh...I'm not quite with you.

JSON: I'm JavaScript's object notation, which means I'm *everywhere*.

Head First: Sorry, but you've completely lost me.

JSON: The first two letters are the key ones: I'm a JavaScript standard, which means you'll find me everywhere JavaScript is...which means I'm in every major web browser on the planet.

Head First: What's that got to do with Python?

JSON: That's where the other two letters come into play. Because I was initially designed to allow JavaScript data objects to be transferred from one JavaScript program to another, I've been extended to allow objects to be transferred regardless of what programming language is used to create the data. By using the JSON library provided by your favorite programming language, you can create data that is interchangeable. If you can read a JSON data stream, you can recreate data as you see fit.

Head First: So I could take an object in, say,

Python, use JSON to convert it to JSON's object notation, and then send the converted data to another computer running a program written in C#?

JSON: And as long as C# has a JSON library, you can recreate the Python data as C# data. Neat, eh?

Head First: Yes, that sounds interesting...only [winks] why would anyone in their right mind want to program in C#?

JSON: [laughs] Oh, come on now: *be nice*. There's plenty of reasons to use different programming languages for different reasons.

Head First: Which goes some of the way to explain why we have so many great programming titles, like *Head First C#*, *Head First Java*, *Head First PHP* and *MySQL*, *Head First Rails*, and *Head First JavaScript*.

JSON: Was that a shameless, self-serving plug?

Head First: You know something...I think it might well have been! [laughs].

JSON: [laughs] Yes, it pays to advertise.

Head First: And to share data, right?

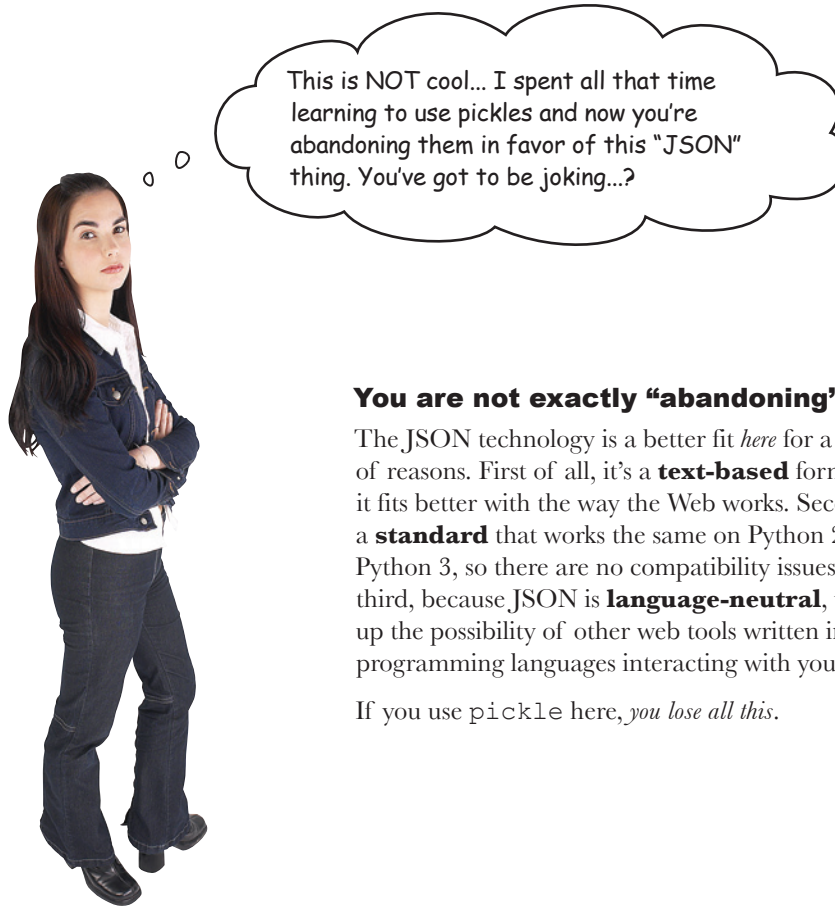
JSON: Yes! And that's exactly my point: when you need a *language-neutral data interchange format* that is easy to work with, it's hard to pass me by.

Head First: But how can you be "language neutral" when you have *JavaScript* in your name?

JSON: Oh, that's just my name. It's what they called me when the only language I supported was JavaScript, and it kinda stuck.

Head First: So they should really call you something else, then?

JSON: Yes, but "WorksWithEveryProgrammingLanguageUnderTheSunIncludingPythonObjectNotation" doesn't have quite the same ring to it!



You are not exactly “abandoning” pickle.

The JSON technology is a better fit *here* for a number of reasons. First of all, it’s a **text-based** format, so it fits better with the way the Web works. Second, it’s a **standard** that works the same on Python 2 and Python 3, so there are no compatibility issues. And third, because JSON is **language-neutral**, you open up the possibility of other web tools written in other programming languages interacting with your server.

If you use `pickle` here, *you lose all this*.



An IDLE Session

JSON is an established web standard that comes preinstalled with Python 2 and Python 3. The JSON API is not that much different to the one used by `pickle`:

```
>>> import json
>>> names = ['John', ['Johnny', 'Jack'], 'Michael', ['Mike', 'Mikey', 'Mick']]
>>> names
['John', ['Johnny', 'Jack'], 'Michael', ['Mike', 'Mikey', 'Mick']]

>>> to_transfer = json.dumps(names)
>>> to_transfer
'["John", ["Johnny", "Jack"], "Michael", ["Mike", "Mikey", "Mick"]]'

>>> from_transfer = json.loads(to_transfer)
>>> from_transfer
['John', ['Johnny', 'Jack'], 'Michael', ['Mike', 'Mikey', 'Mick']]

>>> names
['John', ['Johnny', 'Jack'], 'Michael', ['Mike', 'Mikey', 'Mick']]
```

Import the JSON library.

Create a list of lists.

Transform the Python list-of-lists into a JSON list of lists.

The format is similar, but different.

Transform the JSON list of lists back into one that Python understands.

The new data is exactly the same as the original list of lists.



Sharpen your pencil

Add a new function to the `athletemodel` module that, when called, returns the list of athlete names as a string.

Call the new function `get_names_from_store()`.

```
.....
.....
.....
.....
.....
.....
```



Sharpen your pencil Solution

You were to add a new function to the `athletemodel` module that, when called, returns the list of athlete names as a string.

You were to all the new function `get_names_from_store()`.

Extract a list
of athlete names
from the data.

```
def get_names_from_store():
```

```
    athletes = get_from_store()
```

```
    response = [athletes[each_ath].name for each_ath in athletes]
```

```
    return(response)
```

Get all the data from the pickle.

Return the list to the caller.



So...rather than running a CGI script to create a HTML web page, you want me to deliver just the data, right? That's OK. Not a problem—just be sure to tell me which script to run...



Exercise

With your new function written and added to the `athletemodel` module, create a new CGI script that, when called, returns the data from the `get_names_from_store()` function to the web requester as a JSON data stream.

Call your new script `cgi-bin/generate_names.py`.

Hint: Use `application/json` as your Content-type.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....



I may be small, but I'm mighty capable. Whether you need a web page or just your data, you can count on me to get the job done.



Exercise Solution

With your new function written and added to the `athletemodel` module, you were to create a new CGI script that, when called, returns the data from the `get_names_from_store()` function to the web requester as a JSON data stream.

You were to call your new script `cgi-bin/generate_names.py`.

Don't forget this "magic" first line if you're running on Linux or Mac OS X.

→ `#!/usr/local/bin/python3`

`import json`

`import athletemodel`

`import yate`

← Do your imports.

`names = athletemodel.get_names_from_store()`

← Get the data from the model.

Start with the appropriate "Content-type": line.

→ `print(yate.start_response('application/json'))`

`print(json.dumps(sorted(names)))`

← Sort "names", then convert to JSON and send to STDOUT.



Watch it!

Take care testing your JSON-generating CGI code.

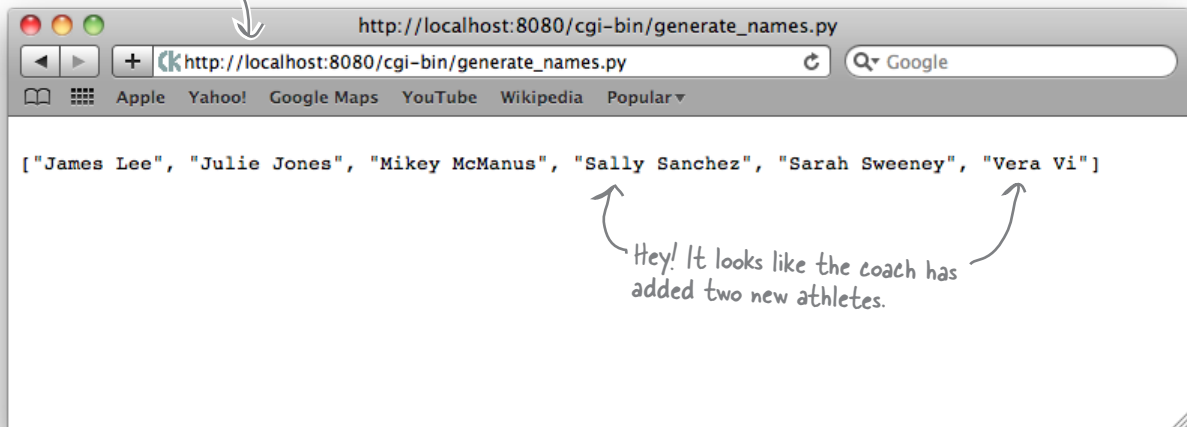
The behavior you see when testing your JSON-generating CGI script will differ depending on the web browser you are using. For instance, Firefox might attempt to download the generated data as opposed to display it on screen.



Test Drive

If it is not already running, start your web server and be sure to set the executable bit with the `chmod +x cgi-bin/generate_names.py` command (if on Linux or Mac OS X). When you're ready, grab your favorite web browser and take your new CGI for a spin.

Enter the web address of the CGI in your browser's location bar.



The web server's logging information confirms that the CGI executed.

```
File Edit Window Help GeneratingJSON
$ python3 simple_httpd.py
Starting simple_httpd on port: 8080
localhost - - [18/Sep/2010 06:31:29] "GET /cgi-bin/generate_names.py HTTP/1.1" 200 -
localhost - - [18/Sep/2010 06:35:29] "GET /cgi-bin/generate_list.py HTTP/1.1" 200 -
localhost - - [18/Sep/2010 06:35:35] "POST /cgi-bin/generate_timing_data.py HTTP/1.1" 200 -
localhost - - [18/Sep/2010 06:35:38] "GET /cgi-bin/generate_list.py HTTP/1.1" 200 -
localhost - - [18/Sep/2010 06:35:40] "GET /index.html HTTP/1.1" 200 -
localhost - - [18/Sep/2010 06:35:49] "GET /cgi-bin/generate_names.py HTTP/1.1" 200 -
```

That worked!

Now all you have to do is arrange for the Android emulator to request the data within a Python script and display the list of names on the smartphone's screen. *How hard can that be?*

The SL4A Android API

The SL4A technology provides a high-level API to the low-level Android API, and SL4A's API is documented in the online API reference:

<http://code.google.com/p/android-scripting/wiki/ApiReference>

Recall the code from earlier, which demonstrated a minimal Android SL4A app:

Import the "android" library and create a new app object.

Create an appropriate message and display it on screen.

```
import android
app = android.Android()
msg = "Hello from Head First Python on Android"
app.makeToast(msg)
```

Six calls to the Android API let you create a list of selectable items in a dialog, together with *positive* and *negative* buttons, which are used to indicate the selection your user made. Note how each of the calls to the Android "dialog" API results in something appearing on screen.

Always start with an import.

```
import android
```

```
app = android.Android()
```

Create an Android app object.

```
app.dialogCreateAlert("Select an athlete:")
```

```
app.dialogSetSingleChoiceItems(['Mikey', 'Sarah', 'James', 'Julie'])
```

```
app.dialogSetPositiveButton("Select")
```

```
app.dialogSetNegativeButton("Quit")
```

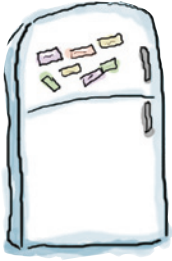
```
app.dialogShow()
```

```
resp = app.dialogGetResponse().result
```

Display your dialog on the phone.

Wait for a response from your user.





Android Code Magnets

Here is the code to a program that queries your web server for the list of names as a JSON array and then displays the list on the smartphone. The only trouble is, the second half of the program is a bunch of mixed-up code magnets at the bottom of the screen. Your job is to rearrange the magnets to complete the program.

```
import android
import json
import time
```

```
from urllib import urlencode
from urllib2 import urlopen
```

All of this program's messages are in one place.

The name of the CGI script to run on the web server

```
hello_msg = "Welcome to Coach Kelly's Timing App"
list_title = 'Here is your list of athletes:'
quit_msg = "Quitting Coach Kelly's App."
web_server = 'http://192.168.1.33:8080'
get_names_cgi = '/cgi-bin/generate_names.py'
```

```
def send_to_server(url, post_data=None):
    if post_data:
        page = urlopen(url, urlencode(post_data))
    else:
        page = urlopen(url)
    return (page.read().decode("utf8"))
```

Change this to the web address that's running your web server.

This function takes both a web address (url) and some optional data (post_data) and sends a web request to your web server. The web response is returned to the caller.

This code's a mess...can you fix it?

```
athlete_names = sorted(json.loads(send_to_server(web_server + get_names_cgi)))
```

```
status_update(quit_msg)
```

```
app.dialogShow()
```

```
def status_update(msg, how_long=2):
    app.makeToast(msg)
    time.sleep(how_long)
```

```
app.dialogSetNegativeButtonText('Quit')
```

```
resp = app.dialogGetResponse().result
```

```
app.dialogCreateAlert(list_title)
```

```
status_update(hello_msg)
```

```
app.dialogSetPositiveButtonText('Select')
```

```
app = android.Android()
```

```
app.dialogSetSingleChoiceItems(athlete_names)
```



Android Code Magnets Solution

Here is the code to a program that queries your web server for the list of names as a JSON array and then displays the list on the smartphone. The only trouble is, the second half of the program is a bunch of mixed-up code magnets at the bottom of the screen. Your job was to rearrange the magnets to complete the program.

```
import android
import json
import time
from urllib import urlencode
from urllib2 import urlopen

hello_msg      = "Welcome to Coach Kelly's Timing App"
list_title     = 'Here is your list of athletes:'
quit_msg       = "Quitting Coach Kelly's App."
web_server     = 'http://192.168.1.33:8080'
get_names_cgi  = '/cgi-bin/generate_names.py'
```

```
def send_to_server(url, post_data=None):
    if post_data:
        page = urlopen(url, urlencode(post_data))
    else:
        page = urlopen(url)
        return(page.read().decode("utf8"))
```

Create an Android
app object.

```
app = android.Android()
```

```
def status_update(msg, how_long=2):
    app.makeToast(msg)
    time.sleep(how_long)
```

Say
"hello".

```
status_update(hello_msg)
```

This is a little
function for
displaying short
messages on the
phone.

Send the web request
to your server, then
turn the JSON response
into a sorted list.

```
athlete_names = sorted(json.loads(send_to_server(web_server + get_names_cgi)))
```

```
app.dialogCreateAlert(list_title)
```

```
app.dialogSetSingleChoiceItems(athlete_names)
```

```
app.dialogSetPositiveButtonText('Select')
```

```
app.dialogSetNegativeButtonText('Quit')
```

```
app.dialogShow()
```

```
resp = app.dialogGetResponse().result
```

```
status_update(quit_msg)
```

Create a two-buttoned
dialog from the list of
athlete names.

Wait for the user to tap a button,
then assign the result to "resp".

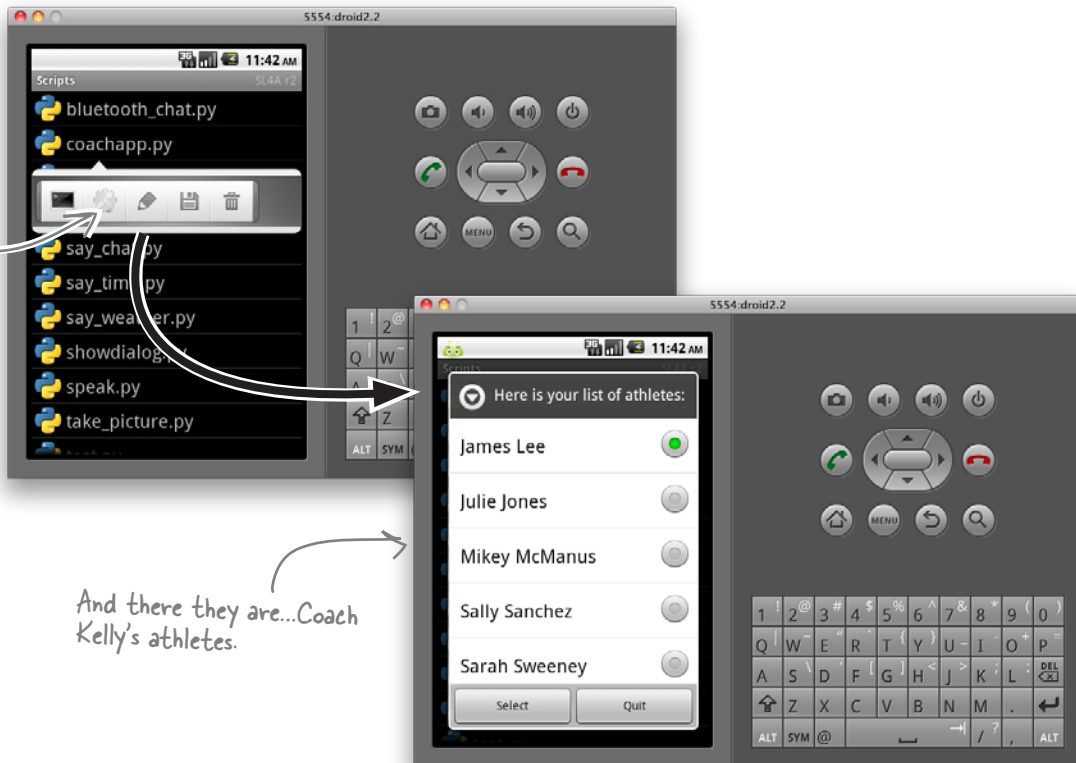
Say "bye bye."



Test Drive

Recall that (for now) your Android Python scripts run within the emulator, not within IDLE. So use the `tools/adb` program to copy your program to the emulator. Call your program `coachapp.py`. When the code is copied over, start SL4A on your emulator, and then tap your script's name.

Tap your app's name, and then tap the "run wheel."



And there they are...Coach Kelly's athletes.

This is looking really good! Your app has communicated with your web server, requested and received the list of athlete names, and displayed the list on your emulator.



If your app doesn't run, don't panic. Check your code for typos.

Run your app again in the Python terminal by tapping on the little terminal icon to the left of the "run wheel" within SL4A. If your code raises an error, you'll see any messages on the emulator's screen, which should give you a good idea of what went wrong.

Select from a list on Android

When your user taps on a button, the “result” of the call to `dialogGetResponse()` is set to `positive` if the first button is tapped or `negative` if the second button is tapped. In your code, you can check the value of `resp`, which is a dictionary, and the `which` key is set to either `positive` or `negative`.

A subsequent call to `dialogGetSelectedItems()` returns the index value of the selected list item.



So...if the positive button is tapped, you can index into the list of athlete names to see which athlete was selected from the displayed list. The selected name can then be sent to the web server to request the rest of the athlete’s data using the `send_to_server()` function.

You can use this behavior in the next version of your code.



Sharpen your pencil

- 1 Assume that you have a CGI script called `cgi-bin/generate_data.py`, which, when called, requests the data for a named athlete from the server.

Provide the code (which includes a call to `thensend_to_server()` function) to implement this functionality:

- 2 Additionally, write the code required to display the list of times returned from the server within an Android dialog.

Hints: Use the `dialogSetItems()` method from the Android API to add a list of items to a dialog. Also, remember that the data arriving over the Internet will be formatted using JSON.



Sharpen your pencil

Solution

- 1** You were to assume that you have a CGI script called `cgi-bin/generate_data.py`, which, when called requests the data for a named athlete from the server.

Provide the name of the CGI to run.

You were to provide the code (which includes a call to the `send_to_server()` function) to implement this functionality:

```
get_data_cgi = '/cgi-bin/generate_data.py'
```

Send the request to the web server, together with the athlete name.

```
send_to_server(web_server + get_data_cgi, {'which_athlete': which_athlete})
```

Include the data.

- 2** Additionally, you were to write the code required to display the list of times returned from the server within an Android dialog:

Which button was pressed?

When your user taps the "positive" button...work out the index value chosen.

```
if resp['which'] in ('positive'):
```

```
    selected_athlete = app.dialogGetSelectedItems().result[0]
```

The index value is in the first element of the list of results returned from the dialog.

Look up the athlete's name using the index value.

```
    which_athlete = athlete_names[selected_athlete]
```

```
    athlete = json.loads(send_to_server(web_server + get_data_cgi,
                                       {'which_athlete': which_athlete}))
```

Send a new web request to the server to fetch the athlete's data.

Dynamically create the dialog's title.

```
    athlete_title = which_athlete + ' top 3 times:'
```

```
    app.dialogCreateAlert(athlete_title)
```

The user needs to see only the data this time, so you need to use "dialogSetItems()".

```
    app.dialogSetItems(athlete['Top3'])
```

```
    app.dialogSetPositiveButtonText('OK')
```

```
    app.dialogShow()
```

Set the single button's text.

Wait for a tap from the user.

```
    resp = app.dialogGetResponse().result
```

The athlete's data CGI script

Here's the code for the `cgi-bin/generate_data.py` CGI script, which takes a web request and returns the indicated athlete's data from the model:

```

#!/usr/local/bin/python3

import cgi
import json
import athletemodel
import yate

athletes = athletemodel.get_from_store()
form_data = cgi.FieldStorage()
athlete_name = form_data['which_athlete'].value
print(yate.start_response('application/json'))
print(json.dumps(athletes[athlete_name]))
  
```

Process the data sent with the request and extract the athlete's name.

Get all the data from the model.

Start a web response, with JSON as the data type.

Include the indicated athlete's data in the web response, formatted by JSON.

The complete Android app, so far

You've made quite a few changes to your program at this stage. Before you test it on the Android emulator, take a moment to look at your code *in its entirety*:

```

import android
import json
import time

from urllib import urlencode
from urllib2 import urlopen

hello_msg      = "Welcome to Coach Kelly's Timing App"
list_title     = 'Here is your list of athletes:'
quit_msg       = "Quitting Coach Kelly's App."

web_server     = 'http://192.168.1.34:8080'

get_names_cgi  = '/cgi-bin/generate_names.py'
get_data_cgi   = '/cgi-bin/generate_data.py'
  
```

The rest of your code is on the following page.

```

def send_to_server(url, post_data=None):
    if post_data:
        page = urlopen(url, urlencode(post_data))
    else:
        page = urlopen(url)
    return (page.read().decode("utf8"))

app = android.Android()

def status_update(msg, how_long=2):
    app.makeToast(msg)
    time.sleep(how_long)

status_update(hello_msg)

athlete_names = sorted(json.loads(send_to_server(web_server + get_names CGI)))

app.dialogCreateAlert(list_title)
app.dialogSetSingleChoiceItems(athlete_names)
app.dialogSetPositiveButtonText('Select')
app.dialogSetNegativeButtonText('Quit')
app.dialogShow()
resp = app.dialogGetResponse().result

if resp['which'] in ('positive'):
    selected_athlete = app.dialogGetSelectedItems().result[0]
    which_athlete = athlete_names[selected_athlete]
    athlete = json.loads(send_to_server(web_server + get_data CGI,
                                       {'which_athlete': which_athlete}))

    athlete_title = athlete['Name'] + ' (' + athlete['DOB'] + '), top 3 times:'
    app.dialogCreateAlert(athlete_title)
    app.dialogSetItems(athlete['Top3'])
    app.dialogSetPositiveButtonText('OK')
    app.dialogShow()
    resp = app.dialogGetResponse().result

status_update(quit_msg)

```

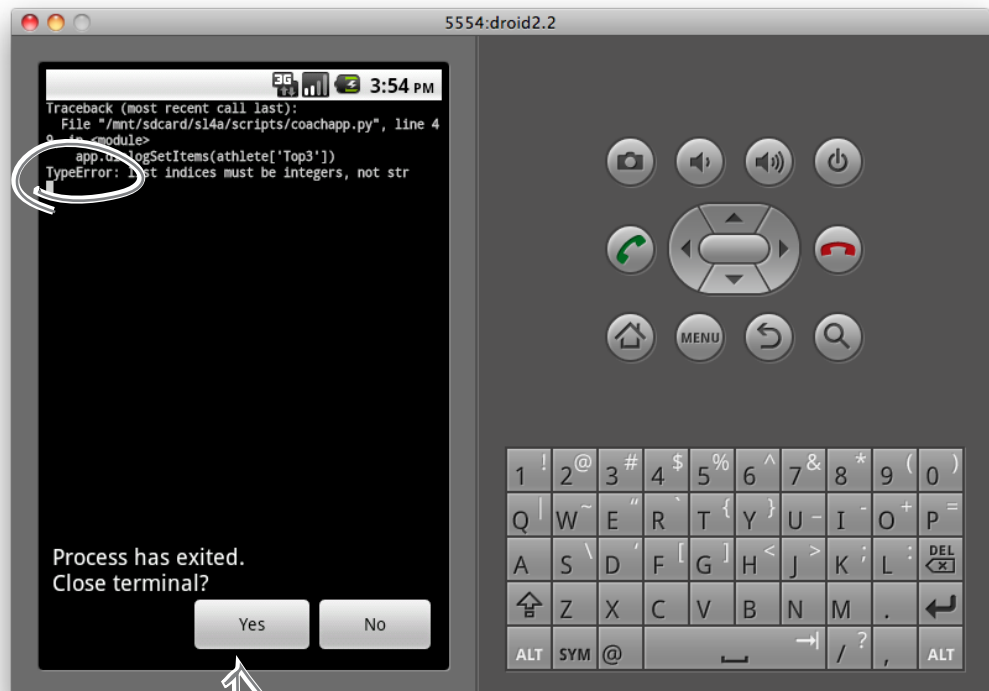


Test Drive

Let's give the latest version of your app a go. Copy the app to your emulator, and put the new CGI script in your `cgi-bin` folder on your web server (remember to set the executable bit, if needed). What happens when you run your latest app using the emulator's Python shell as opposed to the "run wheel"?

You are dumped into the Python shell with a rather nasty error message.

You're getting a "TypeError".



After reading the error message, click "Yes" to return to the SL4A script listing.

Yikes! Your code has a `TypeError`, which is crashing your app when you try to display the selected athlete's timing data. *Why do you think this is happening?*

The data appears to have changed type



Let's add a debugging line of code to your CGI script to try and determine what's going on. Recall that the CGI mechanism captures any output your script sends to *standard output* by default, so let's use code like this to send your debugging message to the web server's console, which is displaying on *standard error*:

Import "sys" from the standard library. →

```
import sys

print(json.dumps(athletes[athlete_name]), file=sys.stderr)
```

Redirect the output from "print()" to "stderr", rather than the default, which is "stdout".

Run your app again and, of course, it's still crashes with a `TypeError`. However, if you check your web server's console screen, you'll see that the data being sent as the JSON web response is clearly visible. *Notice anything?*

This is a list of athlete timing values...but where's the name and DOB values? →

```
File Edit Window Help JustWhatsInTheData
$ python3 simple_httpd.py
Starting simple_httpd on port: 8080
192.168.1.33 - - [18/Sep/2010 17:40:04] "GET /cgi-bin/generate_names.py HTTP/1.1" 200 -
192.168.1.33 - - [18/Sep/2010 17:40:08] "POST /cgi-bin/generate_data.py HTTP/1.1" 200 -
["2-44", "3:01", "2.44", "2.55", "2.51", "2:41", "2:41", "3:00", "2-32", "2.11", "2:26"]
```

JSON can't handle your custom datatypes

Unlike `pickle`, which is smart enough to pickle your custom classes, the JSON library that comes with Python isn't. This means that the standard library's JSON library can work with Python's built-in types, but not with your `AthleteList` objects.

The solution to this problem is straightforward: add a method to your `AthleteList` class to convert your data into a dictionary, and send *that* back to the app. Because JSON supports Python's dictionary, this should work.



Exercise

Let's create a new method in your `AthleteList` class. Called `to_dict()`, your new method needs to convert the class's attribute data (`name`, `DOB`, and `top3`) into a dictionary. Be sure to decorate your new method with `@property`, so that it appears to be a new attribute to users of your class.

.....

.....

.....

.....

.....

.....

.....

there are no Dumb Questions

Q: What's the purpose of this `@property` thing again?

A: The `@property` *decorator* lets you specify that a method is to be presented to users of your class *as if it were an attribute*. If you think about things, your `to_dict()` method doesn't change the state of your object's data in any way: it merely exists to return the object's attribute data as a dictionary. So, although `to_dict()` is a method, it behaves more like an attribute, and using the `@property` decorator lets you indicate this. Users of your class (that is, other programmers) don't need to know that when they access the `to_dict` attribute they are in fact running a method. All they see is a **unified interface**: attributes access your class's data, while methods manipulate it.



Exercise Solution

Let's create a new method in your `AthleteList` class. Called `to_dict()`, your new method needs to convert the class's attribute data (`name`, `DOB`, and `top3`) into a dictionary. Be sure to decorate your new method with `@property`, so that it appears to be a new attribute to users of your class.

Decorate your new method with "`@property`".

`@property`

`def as_dict(self):`

Create a new method.

`return({'Name': self.name,`

`'DOB': self.dob,`

`'Top3': self.top3})`

Return a dictionary of the object's data attributes.

Did you remember to use "self"?



As well as updating your `AthleteList` class code, be sure to change `cgi-bin/generate-data.py` to return a dictionary, rather than the object instance, when servicing its web request.

While you're making changes, adjust the `coachapp.py` app code to include the athlete's name and `DOB` values in the second dialog's title.



Test Drive

With your changes applied to `AthleteList.py`, `cgi-bin/generate_data.py` and `coachapp.py`, use the `adb` tool to copy the latest version of your app to the emulator. Let's see how things work now.

Here's the code
that your app uses
in response to an
athlete selection.

```
coachapp.py - /Users/barryp/HeadFirstPython/chapter8/coachapp.py

if resp['which'] in ('positive'):
    selected_athlete = app.dialogGetSelectedItems().result[0]
    which_athlete = athlete_names[selected_athlete]
    athlete = json.loads(send_to_server(web_server + get_data_cgi,
                                     {'which_athlete': which_athlete}))

    athlete_title = athlete['Name'] + ' (' + athlete['DOB'] + '), top 3 times:'
    app.dialogCreateAlert(athlete_title)
    app.dialogSetItems(athlete['Top3'])
    app.dialogSetPositiveButtonText('OK')
    app.dialogShow()
    resp = app.dialogGetResponse().result

status_update(quit_msg)
```



Success.

Your app displays the selected athlete's top three times on screen. *How cool is that?*

Run your app on a real phone

Now that your app is running successfully on your emulator, it's time to try it on a real phone. This is where things get *interesting*.

There are many options when it comes to copying your code to a real device:

- Use file transfer over Bluetooth.
- Use file transfer with a USB connection.
- Use the Android SDK's `adb` tool with USB.
- Use a file transfer tool over WiFi.

Unfortunately, which technique to use (and which work) depends very much on your phone.

At Head First Labs, we've had the greatest and most consistent success with the last option: *use a file transfer tool over WiFi*.



These instructions do not work on the emulator.

*The Android emulator does not currently support Google's **Android Market**, which you'll need access to use when following along with the instructions on these pages.*

Step 1: Prepare your computer

To transfer files securely between your Android phone and your computer, enable SSH file transfers by running an **SSH server** on your computer. How you do this depends on the operating system you are running:

- **Windows:** download one of the many free SSH servers.
- **Mac OS X:** enable remote logins.
- **Linux:** install and enable OpenSSH Server.

Step 2: Install AndFTP on your Android phone

Use the Android Market on your phone to find and install the **AndFTP** app. This excellent tool lets you transfer files to and from your Android phone over FTP, SFTP, and FTPS.

To use it with the SSH server running on your computer, you'll want to select **SFTP** as the file transfer protocol within the app, because **AndFTP** defaults to using the FTP protocol.

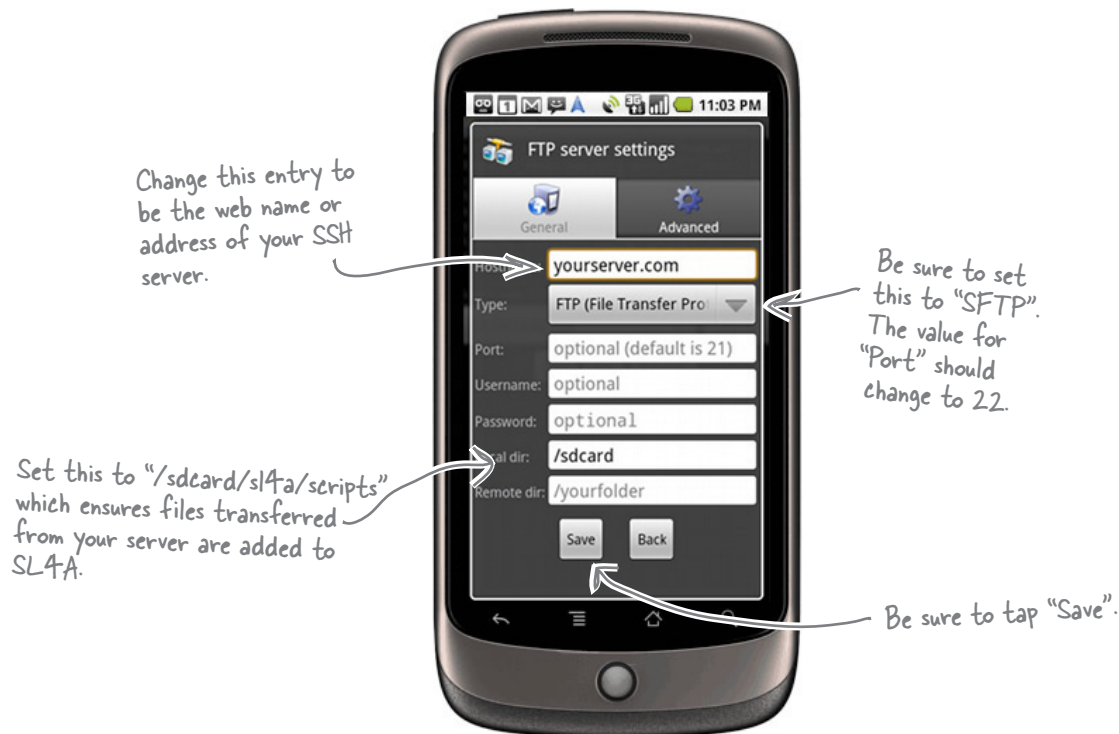
Let's take a look at what's involved.



↗ The AndFTP app is one of our faves.

Configure AndFTP

With AndFTP running on your phone, configure it to connect to your computer (*Hostname*) using SFTP as the transfer protocol (*Type*). Leave the *Port*, *Username*, *Password*, and *Remote dir* entries as they are, but change the *Local dir* entry to `/sdcard/sl4a/scripts`.



With the connection set up, tap AndFTP's Connect button to establish a connection to your SSH server, entering your Username and Password when prompted.

With the connection to the server established, navigate to the server folder containing the file(s) you want to transfer to the phone, mark the files for download, and tap the Download button.

When the download completes, click Disconnect to terminate the connection between the phone and your computer. If you transferred a Python program, it should now be added to the list of scripts within SL4A.

It's time to let Coach Kelly take a look.



The coach is thrilled with his app



Welcome to the future!

You've delivered a solution that automates interaction with your website while providing a modern interface on an Android phone. Your app allows your users to access web data *directly* on their mobile device.

The fact that your server code runs on Python 3 and your Android client code runs on Python 2 makes very little difference: *it's all just Python code, after all.*

All that's left to do is write some code to satisfy Coach Kelly's latest request, and you'll get to that in the next chapter.

This is great work.



Your Python Toolbox

You've got Chapter 8 under your belt and you've added some key Python techniques to your toolbox.

Python Lingo

- "Python 2" – the previous release of Python, which has compatibility "issues" with Python 3 (and are not worth getting worked up over).

Android Lingo

- "SL4A" – the Scripting Layer for Android lets you run Python on your Android device.
- "AVD" – an Android Virtual Device which lets you emulate your Android device on your computer.



BULLET POINTS

- The `json` library module lets you convert Python's built-in types to the text-based JSON data interchange format.
- Use `json.dumps()` to create a stringed version of a Python type.
- Use `json.loads()` to create a Python type from a JSON string.
- Data sent using JSON needs to have its `Content-Type`: set to `application/json`.
- The `urllib` and `urllib2` library modules (both available in Python 2) can be used to send encoded data from a program to a web server (using the `urlencode()` and `urlopen()` functions).
- The `sys` module provides the `sys.stdin`, `sys.stdout` and `sys.stderr` input streams.

